

Attacks and Remedies for Randomness in AI

Cryptanalysis of PHILOX and THREEFRY

Jens Alich¹, Thomas Eisenbarth², Hossein Hadipour¹, Gregor Leander¹,
Felix Mächtle², Yevhen Perehuda¹, Shahram Rasoolzadeh¹, Jonas Sander², and
Cihangir Tezcan³

¹ Ruhr University Bochum, Bochum, Germany,

² University of Lübeck, Lübeck, Germany,

³ Middle East Technical University, Ankara, Turkey.

Abstract. In this work, we address the critical yet understudied question of the security of the most widely deployed pseudorandom number generators (PRNGs) in AI applications. We show that these generators are vulnerable to practical and low-cost attacks. With this in mind, we conduct an extensive survey of randomness usage in current applications to understand the efficiency requirements imposed in practice. Finally, we present a cryptographically secure and well-understood alternative, which has a negligible effect on the overall AI/ML workloads. More generally, we recommend the use of cryptographically strong PRNGs in all contexts where randomness is required, as past experience has repeatedly shown that security requirements may arise unexpectedly even in applications that appear uncritical at first.

Keywords: PRNG · GPU · PHILOX · THREEFRY · SPECK · differential-linear cryptanalysis · multiplicative differentials

1 Introduction

Machine learning systems are now prevalent in products and scientific workflows [27, 33, 38, 49, 50] with many of their core algorithms heavily depending on randomness during training and inference. Sampling-based decoding in large language models (LLMs) [33], stochastic seeding in image [27] and video [50] generators, and noise for differentially private (DP) training [21] are all driven by pseudorandom number generators (RNGs). As the outputs of these systems (e.g., trained models or generated media) become increasingly targeted by attacks, the demand for robust and cryptographically sound RNGs has surged [15, 39].

Table 1 summarizes the RNGs exposed by widely used machine learning frameworks, i.e., TensorFlow [1], PyTorch [40], JAX [13], PaddlePaddle [3], MXNet [14] and Opacus [51], across common hardware targets. Counter-based generators designed for parallelism, i.e., PHILOX and THREEFRY, dominate accelerator backends, reflecting their suitability for vectorized kernels and reproducible, splittable streams. In contrast, CPU architectures still use MT19937,

Table 1: PRNG algorithms used by major machine learning frameworks across hardware platforms. Abbreviations: P = PHILOX (10 rounds), MT = MT19937, T = THREEFRY (20 rounds), A = AES-128-CTR, LFSR = 64-bit Linear-Feedback Shift Register, X = xoroshiro128aox. A dash (–) denotes no support. Unless noted, seed size is 64 bits. Experimental versions are omitted.

	TensorFlow	PyTorch	JAX	PaddlePaddle	MXNet	Opacus
Github Stars	192K	93.2K	33.4K	23.2K	20.8K	1.8K
Nvidia	P/T	P	T	P	P ^b	P/A ^a
x86	P/T	MT ^b	T	MT	MT ^b	MT ^b /A ^a
Apple M-Series	P/T	P/MT ^b	T	MT	–	MT ^b /P/A ^a
Nvidia Jetson	P/T	P/MT ^b	T	P	P ^b	MT ^b /P/A ^a
Coral Edge TPU	P/T	–	–	–	–	–
AWS Trn/Inf	P ^b /T ^b	P ^b /T ^b	–	–	P ^b /T ^b	–
Intel Gaudi	–	P/LFSR	–	–	–	–
Graphcore IPU	P/T	P/MT ^b /X	T	–	–	–

^a Secure but without seed or a 32-bit seed.

^b The key size is 32 bits.

Xoroshiro variants, and even LFSRs. These algorithms are attractive for their speed, but offer weak or no cryptographic protection. AES-128 appears only in the DP library Opacus and comes with seeding constraints: Either an unseeded secure mode or a 32-bit seed is used. However, disabling seeding is untenable for modern ML workflows: It undermines reproducibility and prevents practitioners from fixing seeds to obtain stable outputs, for example, when iterating on prompts for image generation.

Furthermore, porting a workload between different platforms can silently change its RNG semantics, eroding statistical equivalence and reproducibility across runs. The security posture is inconsistent by default: The same training script might use a robust counter-based stream on one backend and a predictable legacy generator on another. Moreover, the interfaces offered to practitioners conflate very different guarantees under the same abstraction, leaving room for configuration mistakes that negate privacy or integrity assumptions.

In this paper, we argue that contemporary ML stacks need an RNG primitive that is secure by construction and performant under realistic loads.

Unexpected Security Demands. Security expectations around randomness in ML workloads have evolved in ways that early designers of RNGs did not anticipate. Attacks such as prompt stealing in generative models [39] or PRNG-based exploits in randomized smoothing [15] exploit the fact that existing generators are predictable or miss-seeded – issues that were initially considered irrelevant in non-adversarial settings like simulation or graphics. In practice, the randomness infrastructure of ML systems has quietly become part of their security perimeter.

This gap is also reflected in the design philosophy of widely adopted counter-based RNGs. As highlighted in [43], the newly designed PRNGs, PHILOX and THREEFRY were engineered for parallelism, statistical quality, and reproducibility. Their evaluation criterion was *crush-resistance*, that is the ability to pass the stringent BigCrush statistical tests from TestU01 in [37], rather than resistance to adversarial prediction or state compromise. The authors explicitly note that these generators are not cryptographically secure, but merely *good enough* as long as they withstand statistical batteries. What was once a reasonable compromise for HPC or Monte Carlo workloads now falls short when the same generators underpin sensitive ML tasks with adversarial threat models.

In summary, current ML/AI applications reveal, on the one hand, a strong and partly unexpected demand for cryptographically secure randomness, on the other hand, the most widely deployed primitives are rooted in cryptographic ideas but lack public scrutiny, leaving their security properties uncertain.

Our Contribution

In our work, we address the critical yet understudied question of the security of the most widely deployed PRNGs. As we show, these not only fail to deliver the promised security, but are also vulnerable to practical, low-cost attacks. With this in mind, we conduct an extensive survey of randomness usage in current applications to understand the efficiency requirements imposed, before finally presenting a cryptographically secure and well-understood alternative.

Security of Current Standards. We focus on the main and most relevant cases, THREEFRY and PHILOX, proposed in [43]. THREEFRY is based on a known cryptographic primitive, THREEFISH, the block cipher used in the SKEIN hash function, one of five finalists in the NIST hash function competition. In fact, THREEFRY is essentially a round-reduced and shortened version of THREEFISH. PHILOX, on the other hand, is a more non-standard design that mainly uses the multiplication of 32-bit words in a Feistel-like structure as its round function.

The main security problem with both constructions is the far too short key length of 64 bits. This is especially problematic since, being optimized for GPUs (for good reasons), brute-forcing the used key is particularly cheap on exactly those devices. Following the recent work [47], in section 4, we show that recovering the 64-bit key for these ciphers is possible in a single day on 578 RTX 4090 GPUs for 32-bit word sized PHILOX, 1038 RTX 4090 GPUs for 64-bit word sized PHILOX, and 521 RTX 4090 GPUs for THREEFRY. We note that a single RTX 4090 costs around 1900 EUR at the time of writing.

However, the short key length is not the only security shortcoming we found. Both ciphers are vulnerable to attacks with practical time complexity, thereby violating the security properties claimed by their designers. Technically, the attacks are very different in nature and interesting in their own right.

For PHILOX, we present an attack based on multiplicative differentials, a rarely applied technique originally introduced in [12, 24]. Unlike classical XOR-based differentials, multiplicative differentials exploit the algebraic structure of

modular multiplication, which is central to the design of PHILOX. By carefully analyzing the propagation of these differentials through the mulhi and mullo operations, we construct differential-linear distinguishers on reduced-round instances of the cipher. In particular, we identify two distinguishers on 5 rounds of PHILOX- $2 \times W$ with a high correlation and two distinguishers on 6 rounds of PHILOX- $4 \times W$ with probability 1. Moreover, we show that an attack on 7 rounds of PHILOX- $4 \times W$ can be derived using these distinguishers where the entire $2W$ -bit key is recovered with a complexity of about 2^W encryptions requiring $W + 1$ chosen plaintext data.

For THREEFRY, we present a differential-linear distinguisher on reduced-round THREEFRY. Interestingly, this attack vector deviates significantly from the best known attacks on the well-studied cipher THREEFISH, on which THREEFRY is based. Since the construction only uses (modular) key addition every fourth round, differential and linear trail probabilities that stem from multiplying probabilities across different rounds are hard to justify. Moreover, as an Addition-Rotation-Xor (ARX) cipher, the differential and linear hull effects can be expected to be significant. While all of these aspects impose considerable restrictions, our approach using a differential-linear distinguisher successfully circumvents these problems. In particular, we rely on experimental verification of three separate components of the distinguisher: Using simple classical methods for the differential part, the differential-linear part, and the classical Goldreich-Levin algorithm for the linear part.

These insights on both primitives, in particular, show that merely increasing the key size by applying a suitable key-scheduling algorithm does not result in cryptographically secure designs. Given that the current methods fail to provide the security required in practice, there is a clear need for a secure and efficient alternative, ideally based on well-scrutinized primitives.

Efficiency Requirements. To better understand the efficiency requirements imposed by modern ML applications, we investigated how much randomness is required and how the demand is distributed over the application runtime. We run a study including four representative ML workloads, including LLM-based text generation, diffusion-based image generation, text-to-video generation, and privacy-preserving neural network training. As detailed in section 2, we found that depending on the application, the randomness demand is either required once at the beginning or steadily spread over time. Therefore, a suitable RNG should enable low-latency sampling to accommodate both short, high-volume bursts and steady per-step demands. Additionally, we found the overhead induced by currently applied RNGs to be negligible compared to the overall runtime of these ML workloads, leaving a room to strengthen the robustness of the used RNGs without significantly affecting the end-to-end application runtime.

There are GPU-optimized table-based implementations of AES [46], arguably the most conservative choice of a primitive, available. While these provide impressive and competitive runtime performance when a large amount of randomness is required at once, they offer less flexibility, as they occupy almost the entire shared memory of the GPU and thus make context switches rather inefficient.

Therefore, in line with the basic design ideas of `THREEFRY` and `PHILOX`, we instead investigate possible alternatives based on operations that are especially efficient on modern GPUs.

POLSTER: A Suitable and Conservative Proposal. Finally, motivated by the security limitations of `THREEFRY` and `PHILOX`, as well as our analysis of the actual requirements for GPU-based PRNGs, we propose a modified, round-reduced variant of a `SPECK` block cipher [5]. Our design employs a 64-bit block size and, crucially, a 128-bit key, making it a practical and secure alternative to the existing constructions. In section 7, we discuss both the implementation and security aspects of this choice, and introduce `POLSTER`, which is essentially `SPECK-64-128` reduced to 20 rounds with an improved key schedule, while still maintaining a sufficient security margin.

2 Requirements for Randomness in ML

To identify the requirements for a secure RNG used in ML, we start by examining how randomness is consumed across four representative ML workloads: text generation with a large language model (LLM), image generation with a diffusion model, text-to-video generation, and privacy-preserving training with differential privacy. These use cases cover the dominant ways randomness enters large-scale ML systems: sampling from discrete distributions during LLM decoding, seeding a diffusion model, and injecting calibrated noise during training to achieve formal privacy guarantees.

During auto-regressive decoding, an LLM assigns a probability p_i to each possible next token index i , and one token is picked at random according to these probabilities. Two sampling methods are commonly used for the selection. *Inverse-transform sampling* draws a single random value $u \sim \text{Uniform}[0, 1]$ and outputs the first token index i satisfying $\sum_{j < i} p_j < u \leq \sum_{j \leq i} p_j$. The *Gumbel-Max trick* draws independent $\text{Gumbel}(0, 1)$ noises g_i and selects the next token index via $\arg \max_i \{\log p_i + g_i\}$, requiring one random draw per candidate token [31]. In the following evaluation, we adhere to the popular HuggingFace Transformer Library [23], which leverages the latter, meaning the per-token randomness consumption scales with the vocabulary size.

In diffusion-based image and video generation, a high-dimensional noise vector is sampled to seed the generation process [27, 50]. The subsequent denoising steps are essentially deterministic. In differentially private model training, gradients are clipped and then perturbed with additive noise (typically Gaussian) at every optimization step [21].

To quantify these randomness usage patterns of all four techniques, we instrumented each workload with a custom RNG hook that counts and timestamps PRNG calls. Concretely, we generated ten tokens with Google’s Gemma-3 [33], produced images with Stable Diffusion 3.5 Turbo [45], created videos with Wan 2.1 [50], and trained a digit classifier on MNIST [36] using Opacus [51], a DP library based on PyTorch [40]. The cumulative count of draws and their timing are reported in Figure 1.

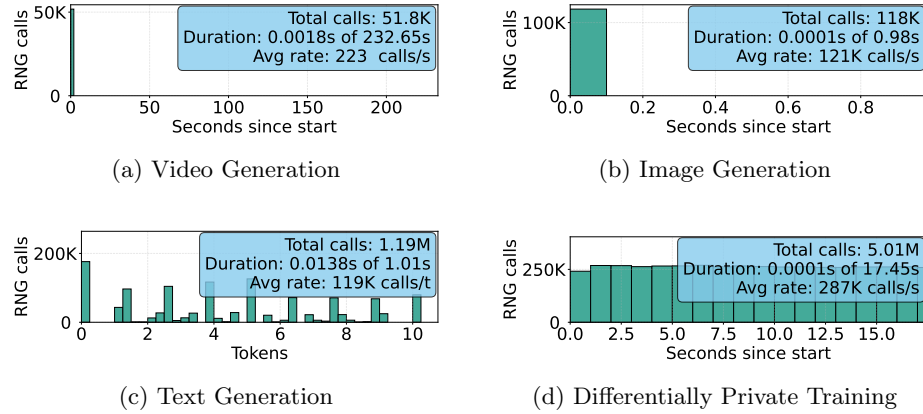


Figure 1: Temporal distribution of PRNG draws across four ML workloads. Measurements use PHILOX (except for the LLM, which uses MT19937 per framework default). Results are averaged over 100 runs on an NVIDIA A100 GPU.

The resulting profiles are consistent with the description above. Diffusion and video models exhibit a sharp spike at the beginning when the initial noise is drawn and afterwards no further randomness is required. In total, the share of the end-to-end runtime attributable to RNG is a small fraction of the total generation time. LLM decoding consumes randomness on every token in the sequence, producing one spike per token as the tokens are sampled. Differentially private training requires fresh randomness on every batch, yielding a fairly constant draw rate over the course of training.

While recent work [15, 39] emphasized the urgent need for cryptographically secure RNGs in ML, the above observations motivate the following additional requirements: The RNG should provide low-latency sampling to accommodate both short, high-volume bursts (e.g., at initialization of image/video generation) and the steady per-step demands of DP and LLM decoding. It should support many independent, reproducible streams across devices, and it should offer deterministic seeding to enable exact replay. Given that random number generation accounts for only a small share of wall-clock time in these workloads, a modest performance overhead is acceptable in exchange for stronger security and reproducibility.

3 Specification of Philox and Threefry

In this section, we specify the PHILOX and THREEFRY block ciphers. We first describe their round functions, followed by the rationale given by the designers to determine the number of rounds.

Throughout the paper, we adopt the convention that the *least significant bit* (LSB) of a binary string is its rightmost entry. Formally, let $X = (x_n, x_{n-1}, \dots,$

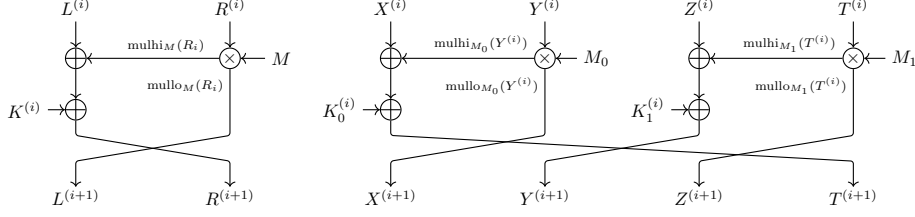


Figure 2: PHILOX-2xW (left) and PHILOX-4xW (right) round functions.

x_2, x_1) denote an n -bit vector, where the indexing begins at 1. Then the LSB is x_1 , which we denote by $\text{LSB}(X) = x_1$.

3.1 Philox Family of Block Ciphers

PHILOX is a family of block ciphers that follows a Feistel-like structure, but with an unconventional design choice: the use of integer multiplication within the round function. Each instance of the cipher is denoted by PHILOX- $N \times W$ - R , where N is the number of words (or branches) in the state, W is the bit size of each word and R is the number of rounds. The recommended parameters for N and W are $(N, W) \in \{(2, 64), (4, 64), (4, 32)\}$.

The block size of each variant is $N \times W$ bits, while the key size is only $(N/2) \times W$ bits, i.e., half the block size. Figure 2 illustrates the round functions of PHILOX-2xW (left) and PHILOX-4xW (right).

The core operation of the round function is the multiplication of one state word by a fixed constant. Let $M \in \{0, 1\}^W$ denote this constant. Then, for a state word $X \in \{0, 1\}^W$, the multiplication is defined as

$$\text{mulhi}_M(X) := \lfloor (X \times M) / 2^W \rfloor, \quad \text{mullo}_M(X) := (X \times M) \bmod 2^W,$$

where the high- and low-halves of the multiplication are subsequently used in the round functions.

It is well known that for an odd constant M , the function $\text{mullo}(\cdot, M)$ is a bijection over $\{0, 1\}^W$. We denote its inverse by $\text{mullo}^{-1}(\cdot, M)$.

The round function of PHILOX-2xW operates on a $2W$ -bit state $(L^{(i)}, R^{(i)}) \in \{0, 1\}^{W \times 2}$ together with a W -bit subkey $K^{(i)}$ as follows:

$$L^{(i+1)} = \text{mullo}_M(R^{(i)}), \quad R^{(i+1)} = \text{mulhi}_M(R^{(i)}) \oplus L^{(i)} \oplus K^{(i)}.$$

All round functions use a fixed odd multiplier; for PHILOX-2x64, this constant is $M = \text{0xD2B74407B1CE6E93}$.

The round function of PHILOX-4xW operates on a $4W$ -bit state $(X^{(i)}, Y^{(i)}, Z^{(i)}, T^{(i)}) \in \{0, 1\}^{W \times 4}$ together with two W -bit subkeys, $K_1^{(i)}$ and $K_2^{(i)}$, to compute the next state $(X^{(i+1)}, Y^{(i+1)}, Z^{(i+1)}, T^{(i+1)})$ as follows:

$$\begin{aligned} X^{(i+1)} &= \text{mullo}_{M_1}(Y^{(i)}), & Y^{(i+1)} &= \text{mulhi}_{M_2}(Y^{(i)}) \oplus X^{(i)} \oplus K_1^{(i)}, \\ Z^{(i+1)} &= \text{mullo}_{M_2}(T^{(i)}), & T^{(i+1)} &= \text{mulhi}_{M_1}(T^{(i)}) \oplus Z^{(i)} \oplus K_2^{(i)}. \end{aligned}$$

PHILOX-4×W uses two different fixed odd multipliers in each round function. For PHILOX-4×32, these constants are $M_1 = 0xD2511F53$ and $M_2 = 0xCD9E8D57$; and for PHILOX-4×64, they are

$$M_1 = 0xD2E7470EE14C6C93 \text{ and } M_2 = 0xCA5A826395121157.$$

The key schedule of PHILOX is deliberately simple and well suited for efficient implementation on GPU platforms. The $N/2$ subkeys are updated in each round according to

$$K_j^{(i+1)} = K_j^{(i)} + C_j, \quad \text{with } K_j^{(0)} = K_j,$$

for each $j \in \{1, \dots, N/2\}$, where K_j s are the $N/2$ words of the user key, and C_j s are fixed constants derived from the fractional part of $(\sqrt{3} - 1)$ and the golden ratio. In particular, the designers specify $C_1 = 0xBB67AE8584CAA73B$ and $C_2 = 0x9E3779B97F4A7C15$ for $W = 64$, and $C_1 = 0xBB67AE85$ and $C_2 = 0x9E3779B9$ for $W = 32$.

In the original paper [43], the authors report that, with the recommended multipliers, PHILOX-2×64 achieves *full crush-resistance* with $R = 6$ rounds. Alongside this 6-round configuration, they also propose a 10-round variant, PHILOX-2×64-10, as an alternative with additional safety margin.

For PHILOX-4×W, the designers found that both PHILOX-4×32 and PHILOX-4×64 achieve full crush-resistance with $R = 7$ rounds. However, libraries that implement these ciphers adopt a 10-round configuration by default, in order to provide a conservative security buffer.

3.2 Threefry Family of Block Ciphers

THREEFRY is a family of block ciphers derived from the well-known THREEFISH cipher. As with THREEFISH, THREEFRY is an ARX cipher.

Each instance is denoted by THREEFRY- $N \times W$ - R , where N is the number of words in the state, W is the bit size of each word, and R is the number of rounds. The recommended and used parameters for THREEFRY are $N \in \{2, 4\}$ and $W \in \{32, 64\}$. Accordingly, the block size of each variant is $N \times W$ bits, and also the key size is $N \times W$ bits. Figure 3 depicts the round functions of THREEFRY-2×W (left) and THREEFRY-4×W (right).

The round function of THREEFRY-2×W operates on a $2W$ -bit state $(X^{(i)}, Y^{(i)}) \in \{0, 1\}^{W \times 2}$ as follows:

$$X^{(i+1)} = X^{(i)} + Y^{(i)}, \quad Y^{(i+1)} = X^{(i+1)} \oplus (Y^{(i)} \lll \rho_{i \bmod 8}),$$

where the addition is modulo 2^W , and $\rho_{i \bmod 8}$ is the rotation constant given in Table 2.

The round function of THREEFRY-4×W operates on a $4W$ -bit state $(X^{(i)}, Y^{(i)}, Z^{(i)}, T^{(i)}) \in \{0, 1\}^{W \times 4}$, producing the next state $(X^{(i+1)}, Y^{(i+1)}, Z^{(i+1)}, T^{(i+1)})$ as follows:

$$\begin{aligned} X^{(i+1)} &= X^{(i)} + Y^{(i)}, & Y^{(i+1)} &= Z^{(i+1)} \oplus (T^{(i)} \lll \rho'_{i \bmod 8}), \\ Z^{(i+1)} &= Z^{(i)} + T^{(i)}, & T^{(i+1)} &= X^{(i+1)} \oplus (Y^{(i)} \lll \rho''_{i \bmod 8}). \end{aligned}$$

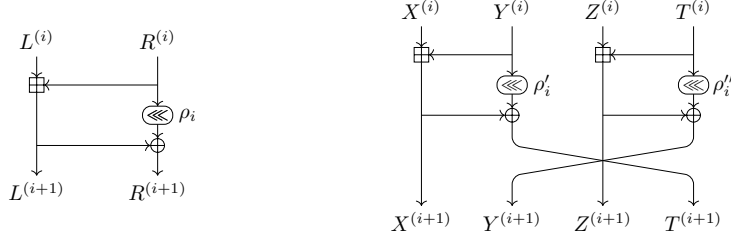
Figure 3: THREEFRY-2 $\times$ W (left) and THREEFRY-4 $\times$ W (right) round functions.

Table 2: The rotation constants used for THREEFRY.

i	0	1	2	3	4	5	6	7
ρ_i	16	42	12	31	16	32	24	21
ρ'_i	10	11	13	23	6	17	25	18
ρ''_i	26	21	27	5	20	11	10	20

Here, all additions are taken modulo 2^W , with (ρ'_i, ρ''_i) denoting the rotation constants specified in Table 2.

The key schedule of THREEFRY follows that of THREEFISH, but is simplified by omitting the *tweak* component, which is considered unnecessary for the PRNG use case on GPU platforms. THREEFRY-N uses N subkey words of W bits: $K_0, \dots, K_{N-1}$. The key schedule then derives an additional key word as

$$K_N = K_0 \oplus K_1 \oplus \dots \oplus K_{N-1} \oplus C,$$

where $C = 0x1BD11BDA$ for $W = 32$ and $C = 0x1BD11BDAA9FC1A22$ for $W = 64$.

Then, at the input of every fourth round, i.e., the round with index $4s$, the N consecutive subkeys $K_s, K_{s+1}, \dots$ are added to the cipher state using modular addition. Specifically, for THREEFRY-2 $\times$ W, the subkeys $(K_s, K_{s+1} + s)$ are used, and for THREEFRY-4 $\times$ W, the subkeys $(K_s, K_{s+1}, K_{s+2}, K_{s+3} + s)$ are employed, where the indices are taken modulo $N + 1$.

This implies that the first key addition uses (K_0, K_1) or (K_0, K_1, K_2, K_3) to the plaintext words. Finally, after R rounds (with R being a multiple of 4), the ciphertext is obtained by performing one additional key addition to the final state with $(K_{R/4}, K_{R/4+1} + R/4)$ or $(K_{R/4}, K_{R/4+1}, K_{R/4+2}, K_{R/4+3} + R/4)$.

In the original paper [43], the authors report that THREEFRY-2 $\times$ W and THREEFRY-4 $\times$ W achieve full crush-resistance with $R = 13$ and $R = 12$ rounds, respectively. However, they recommend 20-round variants for both THREEFRY-2 $\times$ W and THREEFRY-4 $\times$ W in order to provide an additional security margin.

4 Brute Force on GPUs

In this section, we discuss the cost of an exhaustive search for both PHILOX and THREEFRY.

4.1 Preliminaries on GPUs

Modern GPUs have tens of thousands of cores which provide significant speed-ups for parallel applications when compared with CPUs. This speed-up is more significant when the threads do not communicate with each other since data transfers can be the bottleneck. Brute force attacks on block ciphers and counter (CTR) mode of operation for block ciphers are good examples for applications where threads do not need to communicate with each other.

In a brute force attack, the key space can be divided into disjoint subsets where each thread searches the key in its subset for a captured plaintext-ciphertext pair. In the CTR mode of operation, each thread uses a different counter for a fixed key and this time the counter space can be divided into disjoint subsets. Both of these approaches can be used for random number generation. They might look very similar with the only difference in which one of them increments the key and the other one increments the counter. However, the CTR mode can perform better for random number generation because the key schedule part can be eliminated by performing it only once at the CPU side for the fixed key and transferring the round keys to GPU memory, if the context allows such communication.

The set of hardware and software features of a CUDA device is defined by the device compute capability and GPUs are backward compatible with respect to it. For example, a GPU kernel compiled for compute capability 5.2 can be run on a GPU that has a compute capability of 8.9. The performance of a GPU kernel can change depending on the architecture of the GPU, compute capability, the number of threads used in each block, and the used CUDA SDK. Thus, in this work we compiled our codes for every supported compute capability, many different combinations of thread and block numbers, and many different CUDA SDKs. We obtained the best performance when we used 1024 threads and compute capability 5.2 provided the best results for SPECK. For best occupancy, we chose number of threads and number of blocks as 1024. Thus, each thread performs multiple encryptions when more than 2^{20} keys are tried.

Although PHILOX and THREEFRY have versions with 32-bit or 64-bit words, modern GPU registers are 32-bit and 64-bit integer operations require execution of two 32-bit integer instructions. Thus, doubling the words size does not generally double the performance. For example, when we used the designers' publicly available source codes, we achieved 67.0 GB/s, 68.6 GB/s, and 74.6 GB/s on our GTX860M GPU for THREEFRY-4×32-20, THREEFRY-2×64-20, and THREEFRY-4×64-20, respectively.

4.2 Philox on GPUs

PHILOX uses multiplication operation which expands the word size and doubling the word size for multiplication introduces a significant slowdown on GPUs. For this reason, `_umulhi()` CUDA instruction is used to calculate the most significant 32 bits of the product of the two 32-bit unsigned integers. The least significant 32 bits of the product is calculated simply by multiplying the two 32-bit

unsigned integers. This approach provides better performance than multiplying two 64-bit words.

Similarly, we used `_umul64hi()` CUDA instruction for the 64-bit version of PHILOX to calculate the most significant 64 bits of the product of the two 64-bit unsigned integers. This way we are not required to work with 128-bit integers.

We observed that PHILOX runs significantly faster on newer architectures suggesting that `_umulhi()` and `_umul64hi()` integer intrinsics require less clock cycles on newer CUDA GPUs.

PHILOX have two variants with 64-bit key. We run our experiments to try 2^{45} keys and we observed that our optimizations achieve $2^{38.43}$ keys per second for PHILOX- 4×32 -10 and $2^{37.85}$ keys per second for PHILOX- 2×64 -10 on a single RTX 4090 GPU. Thus, an RTX 4090 GPU can capture the key for PHILOX- 4×32 -10 in 578 days or similarly, 578 RTX 4090 GPUs can capture the key in a day. This number increases to 1038 for PHILOX- 2×64 -10 since using 64-bit integers slows it down on GPUs.

4.3 Threefry on GPUs

We run our experiments to try 2^{45} keys and we observed that our optimizations achieve $2^{38.58}$ keys per second for THREEFRY- 2×32 -20 on a single RTX 4090 GPU. Thus, an RTX 4090 GPU can capture the key in 521 days or similarly, 521 RTX 4090 GPUs can capture the key in a day.

Note that the brute force performance of RTX 4090 is compared for the ciphers AES, DES, KASUMI, KLEIN, PRESENT, SPECK, and TEA3 in [47] and for the ciphers ASCON, CHACHA, and SERPENT in [48]. The best performance was achieved for SPECK-64-96 with $2^{36.72}$ keys per second. Achieving almost four times better speed for PHILOX and THREEFRY shows the effect of simplified round functions and small number of rounds of these algorithms.

5 Cryptographic Analysis of Reduced-Round Philox

In this section we present the security analysis of the PHILOX- $2 \times W$ and PHILOX- $4 \times W$ ciphers against multiplicative differential-linear cryptanalysis. We begin by recalling basic concepts on multiplicative differentials; by applying these concepts we then derive two 5-round distinguishers on PHILOX- $2 \times W$ with high correlations and two 6-round distinguishers on PHILOX- $4 \times W$ that hold with probability 1. Finally, we convert the PHILOX- $4 \times W$ distinguishers into key-recovery attacks on 7-round PHILOX- $4 \times W$ with time complexity about 2^W encryptions and data complexity of $W + 1$ chosen plaintexts.

5.1 On Multiplicative Differentials

Classical differential and linear cryptanalysis typically rely on the propagation of XOR-based differences and linear masks through the round operations of a cipher. Although these techniques are powerful for operations such as modular

addition and XOR, they become much less effective in the presence of modular multiplication. It is well known that a single multiplication modulo 2^W is typically enough to destroy conventional differentials and linear trails, leaving little exploitable structure beyond trivial cases.

To address this limitation, Borisov et al. [12] introduced the concept of multiplicative differentials, which replaces the notion of additive differences (including XOR-based differences) with multiplicative ratios. Here, instead of analyzing the evolution of additive differences, one studies how input-output ratios propagate through modular multiplication. This approach preserves an exploitable structure where classical differentials fail, and it has become an established tool for analyzing ciphers that employ multiplication.

Multiplicative differentials have been successfully applied in the cryptanalysis of several designs. They were first exploited against variants of the IDEA block cipher, whose security relies heavily on modular multiplication. Related ideas have also been used against other primitives that combine integer multiplication and XOR, such as NIMBUS [24]. These results highlight the effectiveness of multiplicative techniques in settings where classical XOR-based analysis is inadequate, and provide the foundation for extending differential and linear methods to such ciphers.

In the following, we summarize several structural properties of modular multiplication that will serve as the basis for constructing differential-linear trails in the analysis of PHILOX block ciphers.

Based on the distributivity of multiplication over additive inverses, we have

$$\text{mullo}_M(-X) \equiv -\text{mullo}_M(X) \bmod 2^W.$$

Using this property, for any M , if the inputs differ by a multiplicative factor $-1 \bmod 2^W$, i.e., the pair $(X, -X \bmod 2^W)$, then their images under mullo_M also differ by $-1 \bmod 2^W$. In particular, the outputs form the pair $(Y, -Y \bmod 2^W)$ with $Y = \text{mullo}_M(X)$. Thus, multiplicative differentials are preserved with probability 1 through the multiplication operations.

Lemma 1. *Let X be an odd integer in $\mathbb{Z}_{2^W}$. Then,*

$$-X \bmod 2^W \equiv X \oplus (2^W - 2).$$

Lemma 2. *Let M be an odd integer in $\mathbb{Z}_{2^W}$. Then, for any non-zero $X \in \mathbb{Z}_{2^W}$, we have*

$$\left\lfloor \frac{X \times M}{2^W} \right\rfloor + \left\lfloor \frac{(-X) \times M}{2^W} \right\rfloor = M - 1.$$

The proofs of the above lemmas can be found in previous works, such as [12], and are therefore omitted here. Using Lemma 2 and translating it into PHILOX operations, we obtain the following: For any non-zero $X \in \{0, 1\}^W$,

$$\text{mulhi}_M(-X) \equiv -\text{mulhi}_M(X) + M - 1 \bmod 2^W.$$

In the following, we use these two lemmas to construct 4-round (multiplicative) differential-linear distinguishers on PHILOX-2 $\times$ W and (multiplicative) 6-round differential-linear distinguishers on PHILOX-4 $\times$ W, both of which can be extended non-linearly and deterministically through one additional round, yielding 5-round and 7-round distinguishers, respectively.

5.2 5-Round Distinguishers on Philox-2 $\times$ W

In the following, we present two multiplicative differential-linear distinguishers on 4-round PHILOX-2 $\times$ W, with significant correlations, which can be further extended deterministically to 5-round distinguishers.

Distinguisher 1. *Let $P = (L^{(0)}, R^{(0)})$ and $\tilde{P} = (\tilde{L}^{(0)}, \tilde{R}^{(0)})$ be two plaintexts for PHILOX-2 $\times$ W, such that $R^{(0)}$ is odd (i.e., $\text{LSB}(R^{(0)}) = 1$), and*

$$\begin{cases} \tilde{R}^{(0)} = R^{(0)} \oplus (2^W - 2), \\ \tilde{L}^{(0)} = L^{(0)} \oplus \text{mulhi}_M(R^{(0)}) \oplus (M - 1 - \text{mulhi}_M(R^{(0)})) . \end{cases}$$

Then, after four rounds the corresponding states $(L^{(4)}, R^{(4)})$ and $(\tilde{L}^{(4)}, \tilde{R}^{(4)})$ satisfy the following relation:

$$\Pr \left[\text{LSB}(L^{(4)}) = \text{LSB}(\tilde{L}^{(4)}) \right] = \frac{M - 1}{2^W} ,$$

where the probability is taken over all values for the round keys and the remaining $2W-1$ bits of plaintext P . Besides, the corresponding states after five rounds $(L^{(5)}, R^{(5)})$ and $(\tilde{L}^{(5)}, \tilde{R}^{(5)})$ satisfy the following relation:

$$\Pr \left[\text{LSB}(F_M(L^{(5)}) \oplus R^{(5)}) = \text{LSB}(F_M(\tilde{L}^{(5)}) \oplus \tilde{R}^{(5)}) \right] = \frac{M - 1}{2^W} ,$$

where $F_M := \text{mulhi}_M \circ \text{mullo}_M^{-1}$.

Distinguisher 2. *Let $P = (L^{(0)}, R^{(0)})$ and $\tilde{P} = (\tilde{L}^{(0)}, \tilde{R}^{(0)})$ be two plaintexts for PHILOX-2 $\times$ 32, such that $\tilde{L}^{(0)} = L^{(0)} \oplus 1$ and $\tilde{R}^{(0)} = R^{(0)}$. That is, the two plaintexts differ only in the LSB of the left word. Then, after four rounds the corresponding states $(L^{(4)}, R^{(4)})$ and $(\tilde{L}^{(4)}, \tilde{R}^{(4)})$ satisfy the following relation:*

$$\Pr \left[\text{LSB}(L^{(4)}) = \text{LSB}(\tilde{L}^{(4)}) \right] \approx 0.56 ,$$

where the probability is taken over all values for the round keys and the remaining $2W-1$ bits of plaintext P .

Besides, the corresponding states after five rounds $(L^{(5)}, R^{(5)})$ and $(\tilde{L}^{(5)}, \tilde{R}^{(5)})$ satisfy the following relation:

$$\Pr \left[\text{LSB}(F_M(L^{(5)}) \oplus R^{(5)}) = \text{LSB}(F_M(\tilde{L}^{(5)}) \oplus \tilde{R}^{(5)}) \right] \approx 0.56 ,$$

where $F_M := \text{mulhi}_M \circ \text{mullo}_M^{-1}$.

The proofs of Distinguisher 1 and Distinguisher 1 are provided in subsection A.1 and in subsection A.2.

Note that although Distinguisher 2 has a lower probability than the one for Distinguisher 1, its input-differential pattern makes it particularly well-suited for counter-based random number generators. More precisely, for the chosen-plaintext pairs in Distinguisher 2, the inputs need to differ only in the LSB of the left branch. That means, if we have the LSB of the left branch in the first plaintext is zero, then the corresponding paired plaintext naturally occurs after 2^W executions of the generator. This property allows us to efficiently collect the required data in practice.

5.3 6-Round Distinguishers on Philox-4×W

In the following, we present two multiplicative differential-linear distinguishers on 5-round PHILOX-4×W with probability 1, which can be further extended to 6-round distinguishers, also with probability 1.

Distinguisher 3. Let $P = (X^{(0)}, Y^{(0)}, Z^{(0)}, T^{(0)})$ and $\tilde{P} = (\tilde{X}^{(0)}, \tilde{Y}^{(0)}, \tilde{Z}^{(0)}, \tilde{T}^{(0)})$ be two plaintexts for PHILOX-4×W, such that $\text{LSB}(T^{(0)}) = \text{LSB}(\tilde{T}^{(0)})$, and

$$\tilde{X}^{(0)} = X^{(0)}, \quad \tilde{Y}^{(0)} = Y^{(0)}, \quad \tilde{Z}^{(0)} = Z^{(0)} \oplus \text{mulhi}_{M_1}(T^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}).$$

Then, the corresponding states after five rounds, i.e., $(X^{(5)}, Y^{(5)}, Z^{(5)}, T^{(5)})$ and $(\tilde{X}^{(5)}, \tilde{Y}^{(5)}, \tilde{Z}^{(5)}, \tilde{T}^{(5)})$ satisfy the following relation:

$$\text{LSB}(Z^{(5)}) = \text{LSB}(\tilde{Z}^{(5)}).$$

Besides, for the corresponding states after six rounds, i.e., $(X^{(6)}, Y^{(6)}, Z^{(6)}, T^{(6)})$ and $(\tilde{X}^{(6)}, \tilde{Y}^{(6)}, \tilde{Z}^{(6)}, \tilde{T}^{(6)})$ satisfy the following relation:

$$\text{LSB}(F_{M_1}(Z^{(6)}) \oplus Y^{(6)}) = \text{LSB}(F_{M_1}(\tilde{Z}^{(6)}) \oplus \tilde{Y}^{(6)}).$$

The proof of Distinguisher 3 is provided in subsection A.3.

Due to the rotational structure of PHILOX-4×W, these distinguishers can be translated into equivalent ones on different branches, as we explain below.

Distinguisher 4. Let $P = (X^{(0)}, Y^{(0)}, Z^{(0)}, T^{(0)})$ and $\tilde{P} = (\tilde{X}^{(0)}, \tilde{Y}^{(0)}, \tilde{Z}^{(0)}, \tilde{T}^{(0)})$ be two plaintexts for PHILOX-4×W, such that $\text{LSB}(Y^{(0)}) = \text{LSB}(\tilde{Y}^{(0)})$, and

$$\tilde{X}^{(0)} = X^{(0)} \oplus \text{mulhi}_{M_0}(Y^{(0)}) \oplus \text{mulhi}_{M_0}(\tilde{Y}^{(0)}), \quad \tilde{Z}^{(0)} = Z^{(0)}, \quad \tilde{T}^{(0)} = T^{(0)}.$$

Then, the corresponding states after five rounds, i.e., $(X^{(5)}, Y^{(5)}, Z^{(5)}, T^{(5)})$ and $(\tilde{X}^{(5)}, \tilde{Y}^{(5)}, \tilde{Z}^{(5)}, \tilde{T}^{(5)})$ satisfy the following relation:

$$\text{LSB}(X^{(5)}) = \text{LSB}(\tilde{X}^{(5)}).$$

Besides, for the corresponding states after six rounds, i.e., $(X^{(6)}, Y^{(6)}, Z^{(6)}, T^{(6)})$ and $(\tilde{X}^{(6)}, \tilde{Y}^{(6)}, \tilde{Z}^{(6)}, \tilde{T}^{(6)})$ satisfy the following relation:

$$\text{LSB}(F_{M_0}(\tilde{X}^{(6)}) \oplus \tilde{T}^{(6)}) = \text{LSB}(F_{M_0}(X^{(6)}) \oplus T^{(6)}).$$

In comparison to Distinguisher 4, due to the position of the input difference, Distinguisher 3 is particularly well-suited for counter-based random number generators. More precisely, the differences in the plaintexts occur only in the two least significant branches, namely the Z and T words. Moreover, it is possible to choose values in the T branch (the least significant word) such that $\text{mulhi}_{M_1}(T^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)})$ to be very small, e.g., 1. This means that, for fixed values of the three left branches with only a single-bit difference in the LSB of Z branch, we can select plaintexts with odd values of T and $\tilde{T}$ satisfying $\text{mulhi}_{M_1}(T^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}) = 1$, which naturally fulfills the requirements of the distinguisher. In other words, within at most 2^{W+1} executions of the generator, we can obtain many plaintext pairs suitable for this distinguisher.

5.4 Key Recovery Attack on 7- Round Philox-4×W

The 6-round distinguishers on PHILOX-4×W can be turned into a key-recovery attack on 7-round PHILOX-4×W by appending one additional round to the distinguisher, as described below. Note that using either distinguishers lead to essentially the same attack with identical complexity. Therefore, we only describe the attack for Distinguisher 3.

Consider a pair of plaintexts $P = (X^{(0)}, Y^{(0)}, Z^{(0)}, T^{(0)})$ and $\tilde{P} = (\tilde{X}^{(0)}, \tilde{Y}^{(0)}, \tilde{Z}^{(0)}, \tilde{T}^{(0)})$ for PHILOX-4×W, such that $\text{LSB}(T^{(0)}) = \text{LSB}(\tilde{T}^{(0)})$, and

$$\tilde{X}^{(0)} = X^{(0)}, \quad \tilde{Y}^{(0)} = Y^{(0)}, \quad \tilde{Z}^{(0)} = Z^{(0)} \oplus \text{mulhi}_{M_1}(T^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}).$$

Based on Distinguisher 3, the states after six rounds, $(X^{(6)}, Y^{(6)}, Z^{(6)}, T^{(6)})$ and $(\tilde{X}^{(6)}, \tilde{Y}^{(6)}, \tilde{Z}^{(6)}, \tilde{T}^{(6)})$ satisfy

$$\text{LSB}(F_{M_1}(\tilde{Z}^{(6)}) \oplus \tilde{Y}^{(6)}) = \text{LSB}(F_{M_1}(Z^{(6)}) \oplus Y^{(6)}).$$

Using the inverse relations for the additional round, we have the following.

$$\begin{cases} Y^{(6)} = \text{mullo}_{M_0}^{-1}(X^{(7)}) \\ \tilde{Y}^{(6)} = \text{mullo}_{M_0}^{-1}(\tilde{X}^{(7)}) \end{cases} \quad \text{and,} \quad \begin{cases} Z^{(6)} = F_{M_1}(Z^{(7)}) \oplus Y^{(7)} \oplus K_1^{(6)} \\ \tilde{Z}^{(6)} = F_{M_1}(\tilde{Z}^{(7)}) \oplus \tilde{Y}^{(7)} \oplus K_1^{(6)} \end{cases}$$

where $K_1^{(6)}$ is one of the relevant subkeys added in this round. Replacing these expressions into the distinguisher condition yields the following constraint on the states after 7 rounds and the unknown subkey:

$$\begin{aligned} & \text{LSB}\left(F_{M_1}(F_{M_1}(Z^{(7)}) \oplus Y^{(7)} \oplus K_1^{(6)})\right) \oplus \text{LSB}\left(F_{M_1}(F_{M_1}(\tilde{Z}^{(7)}) \oplus \tilde{Y}^{(7)} \oplus K_1^{(6)})\right) \\ &= \text{LSB}\left(\text{mullo}_{M_0}^{-1}(X^{(7)})\right) \oplus \text{LSB}\left(\text{mullo}_{M_0}^{-1}(\tilde{X}^{(7)})\right). \end{aligned}$$

This equality is the key filtering equation used in our attack: it depends on the unknown subkey word $K_1^{(6)}$. Given a plaintext pair satisfying the distinguisher input pattern, one can compute the right-hand side of the key-filtering equation. Next, guess a candidate for the subkey $K_1^{(6)}$ and compute the left-hand side.

Table 3: Complexity of key recovery attacks on 7-round PHILOX-4×W.

Cipher	Data [CP]	Time [Enc]	Memory [Blocks]
PHILOX-4×W-7	$d + 1$	$\sim 2^{2W-d}$	$d + 1$
PHILOX-4×32-7	33	$\sim 2^{32}$	33
PHILOX-4×64-7	65	$\sim 2^{64}$	65

For the correct guess the equality holds deterministically, whereas for an incorrect guess it holds with probability only 2^{-1} . Hence, if we check d plaintext pairs, an incorrect key survives all d tests with probability 2^{-d} . We then can exhaustively search the correct value for the user key, using the remaining candidates for $K_1^{(6)}$ together with the full W -bit value of $K_0^{(6)}$ with a cost of 2^{2W-d} 7-round PHILOX-4×W encryptions.

The key-recovery filtering stage requires, for each candidate $K_1^{(6)}$, evaluating the left-hand side of the key equation on the d ciphertext pairs. Thus, the naive cost of this stage is about $d \cdot 2^W$ evaluations of the left-hand expression. Although this stage admits typical optimizations techniques that can substantially reduce the constant factors, the subsequent exhaustive search over the remaining key material remains the dominant part of the overall time complexity of the attack.

This attack requires d plaintext pairs following the input differential pattern of the distinguisher. Such pairs can be constructed by choosing a single random base plaintext P_0 and then selecting d distinct chosen plaintexts P_i for $1 \leq i \leq d$, so that each pair (P_0, P_i) satisfies the input pattern of the distinguisher. In this way, the data complexity reduces to $d + 1$ chosen plaintexts instead of $2d$.

Compared to a brute-force attack on the reduced-round PHILOX-4×W, which has a computational complexity of 2^{2W} encryptions, the above-described attack requires only about 2^W encryptions. This is achieved with a relatively small number of chosen plaintexts, which, as discussed in previous subsection, can be obtained within at most 2^{W+1} executions of the counter-based PRNG.

One may ask whether Distinguisher 1 or Distinguisher 2 can be used within a key-recovery attack on reduced-round PHILOX-2×W. Indeed, the 5-round distinguishers can be turned into key-recovery attacks against 6-round PHILOX-2×W. However, in this setting the attacker must guess the single W -bit key word used by the cipher, which amounts to searching the *entire* key space. Consequently, any advantage of this key recovery attack over a straightforward brute-force search is negligible. For this reason, we omit a detailed description of key recovery on PHILOX-2×W.

6 Cryptographic Analysis of Reduced-Round Threefry

In this section, we propose a new method for searching for differential-linear (DL) distinguishers, and apply it to THREEFRY. We first explain why we need

the new method. Then, we describe the segmented search algorithm. Finally, we present our results on THREEFRY-2 $\times$ 32.

6.1 Differential-Linear Distinguishers

Langford and Hellman introduced differential-linear cryptanalysis in 1994 [35]. The idea is to combine a short high-probability differential with a short high-correlation linear approximation. This combination makes it possible to build a distinguisher for more rounds than either technique alone.

We define the correlation of a binary random variable X as $\text{Cor}(X) = 2\Pr(X = 0) - 1$. We also write $\langle a, b \rangle$ for the bitwise dot product over $\mathbb{F}_2$. Assume that E is a block cipher and $C = E_K(P)$. Algorithm 1 shows the workflow of a DL distinguisher.

As shown in Figure 4, we decompose E into three segments $E = E_\ell \circ E_m \circ E_u$, where E_u , E_m , and E_ℓ consist of r_u , r_m , and r_ℓ rounds, respectively. Suppose that the following hold: E_u admits a differential, E_ℓ admits a linear approximation, and E_m admits a combined differential-linear distinguisher, such that:

$$\begin{aligned} \Pr_{P,K}(E_u(P) \oplus E_u(P + \Delta_u) = \Delta_m) &= p, \\ \text{Cor}_{X,K}(\langle \Gamma_m, X \rangle \oplus \langle \Gamma_o, E_\ell(X) \rangle) &= q, \\ \text{Cor}_{X,K}(\langle \Gamma_m, E_m(X) \oplus E_m(X + \Delta_m) \rangle) &= r. \end{aligned}$$

Then the correlation of the differential-linear distinguisher for E is approximated by [4, 11, 26]:

$$\text{Cor}_{P,K}(\langle \Gamma_o, E(P) \oplus E(P + \Delta_u) \rangle) \approx p \cdot r \cdot q^2.$$

Most existing methods for finding DL distinguishers follow a bottom-up approach, where the cipher is broken down into its components (such as S-boxes, modular addition, XOR), and the propagation of differences and masks through these components is modeled [8, 26]. This process requires a detailed understanding of how to model each component and how to combine them to form the full model of the cipher. It is also prone to modeling errors. In addition, modeling some operations, such as modular addition, can be challenging and often requires approximations that affect the accuracy of the results [8]. Another limitation is that most existing models assume statistical independence between rounds and components, which may not hold in practice and can lead to large deviations between the estimated and actual probabilities and correlations.

In contrast, a top-down approach treats the primitive as a black box and tries to find distinguishers by directly analyzing its input-output behavior. The main advantage of this approach is that it does not require detailed knowledge about modeling the internal structure of the cipher. For instance, at EURO-CRYPT 2023, Dinur et al. [19] proposed several top-down methods to identify high-probability statistical properties of cryptographic primitives via surrogate differentiation. However, there are fewer works that apply practical top-down methods to find DL distinguishers.

Algorithm 1: DL distinguisher

Input: $E_K, (\Delta, \Gamma), N, \tau$
Output: Distinguishes Between E_K and an Ideal Cipher

```

1 Initialize  $B[0], B[1]$  to zero;
2 forall  $i = 0, \dots, N - 1$  do
3    $P_1 \xleftarrow{\$} \mathbb{F}_2^n, C_1 \leftarrow E_K(P_1);$ 
4    $P_2 \leftarrow P_1 \oplus \Delta, C_2 \leftarrow E_K(P_2);$ 
5    $B[\langle \Gamma, C_1 \oplus C_2 \rangle] + 1;$ 
6  $Cor \leftarrow \frac{B[0] - B[1]}{N};$ 
7 if  $Cor \geq \tau$  then
8   return “ $E_K$ ”;
9 else
10  return “Ideal cipher”;

```

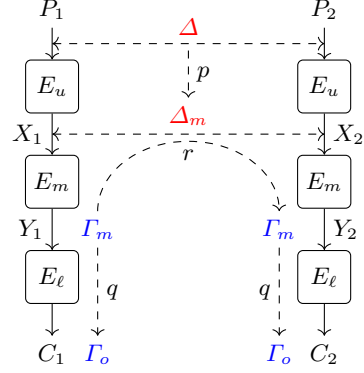


Figure 4: Building blocks of a DL distinguisher in the sandwich framework [26].

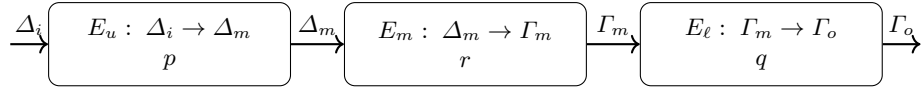


Figure 5: Segmented search for a differential-linear distinguisher. We split E as $E = E_\ell \circ E_m \circ E_u$ and analyze each segment in place, respecting the absolute round boundaries and the key-addition schedule.

6.2 Segmented Search for DL Distinguishers

In this work, we follow a top-down approach to find DL distinguishers by segmenting the cipher into three parts and analyzing each part separately. Our method treats the cipher as an oracle and only calls reduced-round encryption and decryption for the segments under analysis. We do not build MILP or CP models and we do not rely on implementation details beyond the round structure and key-addition schedule. This keeps the method simple, portable across designs, and focused on structurally experimental validated results. We explain our method for finding the DL distinguishers of THREEFRY, but it is general and can also be applied to other primitives.

Segmentation and Method. As shown in Figure 4, we split the full cipher E into three segments E_u , E_m , and E_ℓ . Recent studies on differential-linear cryptanalysis [25, 26], especially on ARX ciphers, show that many strong DL distinguishers follow an hourglass structure. In this structure, the number of active bits shrinks near the input and output of the middle segment E_m and expands again toward the input and output of the full cipher E . We therefore focus on differences Δ_m and masks Γ_m with low Hamming weight. In particular, we restrict our search to single-bit differences and masks at the junctions between segments (i.e., Δ_m and Γ_m).

For analyzing THREEFRY, we start at an absolute round **offset** inside the full-design and respect the key schedule method of the cipher. In THREEFRY, a subkey is added at the beginning and then subkeys are added every four rounds. In our segmentation, we respect these absolute round boundaries and add the subkeys exactly where the design prescribes, so that the combination of the three segments is equivalent to the corresponding reduced-round version of THREEFRY with a given **offset**. In the following steps, expectations and other statistical measures are computed over the indicated random variables, with K ranging over keys and X over states.

Algorithm Flow. The method proceeds in five steps, corresponding to the three segments and the final ranking and verification.

1. *Phase M: Searching for a DL distinguisher on the middle segment.* For each pair (Δ_m, Γ_m) with a single active bit in both Δ_m and Γ_m , we estimate the signed correlation r using N random samples:

$$r := \mathbb{E}_{K,X} \left[(-1)^{\langle \Gamma_m, E_{m,K}(X) \rangle \oplus \langle \Gamma_m, E_{m,K}(X \oplus \Delta_m) \rangle} \right].$$

Here, $\mathbb{E}_{K,X}[Z] = \frac{1}{|\mathcal{K}||\mathcal{X}|} \sum_{K \in \mathcal{K}} \sum_{X \in \mathcal{X}} Z(K, X)$ with K and X sampled uniformly. Equivalently, $\Pr[\langle \Gamma_m, E_{m,K}(X) \rangle = \langle \Gamma_m, E_{m,K}(X \oplus \Delta_m) \rangle] = \frac{1+r}{2}$. We discard pairs (Δ_m, Γ_m) whenever the estimated correlation r falls below a threshold, e.g., $|r| < 2^{-N/2}$, where N denotes the total number of random samples used in the estimation. By the end of this phase, we store a table of tuples $\text{DLCT}[\Delta_m, \Gamma_m] = r$ for the surviving pairs.

2. *Phase U: Searching for a differential distinguisher on the first segment.* For each Δ_m that survives Phase M, we search for an input difference Δ_i that maximizes the probability

$$p = \Pr_{K,Y} [E_{u,K}^{-1}(Y) \oplus E_{u,K}^{-1}(Y \oplus \Delta_m) = \Delta_i].$$

To this end, we sample many random states Y and form pairs $(Y, Y \oplus \Delta_m)$. We then decrypt these pairs backward through the inverse of E_u to obtain inputs (X, X') at the start of E_u , where $X = E_u^{-1}(Y)$, $X' = E_u^{-1}(Y \oplus \Delta_m)$, and $\Delta_i = X \oplus X'$. We estimate p by counting over many randomly chosen pairs $(Y, Y \oplus \Delta_m)$ and keys K . The decryption process mirrors the composed run and applies inverse subkey additions only at the absolute boundaries where the design prescribes key addition. For each Δ_m , we keep the Δ_i that maximizes the observed probability p . At the end of this phase, we obtain a table of tuples $\text{DDT}[\Delta_i, \Delta_m] = p$.

3. *Phase L: Searching for a linear distinguisher on the last segment with Goldreich-Levin.* For each mask Γ_m that survives Phase M, we search for an output mask Γ_o that maximizes

$$q = \left| \mathbb{E}_{K,X} \left[(-1)^{\langle \Gamma_m, X \rangle \oplus \langle \Gamma_o, E_{\ell,K}(X) \rangle} \right] \right|.$$

We define the Boolean oracle $f_K(y) = \langle \Gamma_m, E_{\ell,K}^{-1}(y) \rangle$ on the last segment E_ℓ . A large q indicates that f_K has a heavy Fourier coefficient at frequency Γ_o . The Goldreich–Levin algorithm recovers such coefficients by probing f_K on a small random subspace and applying a fast Walsh–Hadamard transform. For each Γ_m , we run one search on a random key and obtain a candidate Γ_o . Afterwards, we estimate q over several keys to avoid key-dependent artifacts and discard candidates whose average q (across keys) falls below a fixed threshold. Thereby, we obtain a table of tuples $\text{LLT}[\Gamma_m, \Gamma_o] = q$.

4. *Ranking Phase.* We assign a score to each tuple $(\Delta_i, \Delta_m, \Gamma_m, \Gamma_o)$ as an estimate of the overall correlation of the corresponding DL distinguisher:

$$\text{Score} := |\text{Corr}| = p \cdot |r| \cdot q^2.$$

We retain the top k candidates (e.g., $k = 10$) and discard the rest.

5. *Verification Phase.* We recompute p (and q) forward on U (resp. L) with (Δ_i, Δ_m) (resp. (Γ_m, Γ_o)) using a larger sample size and many keys. We then filter candidates with negligible signal by comparing the observed correlation against the noise level induced by the sample size, and we update their scores accordingly. Finally, we reorder the top list based on the updated scores.

Important Details. The segmentation $E = E_\ell \circ E_m \circ E_u$ amplifies the useful signal while keeping each local task computationally manageable. We respect absolute round boundaries and add subkeys exactly where the design prescribes, ensuring that the three phases are semantically identical to the composed run. Restricting input differences and output masks to single active bits at the junctions keeps the search space tractable and aligns with the observation that strong DL distinguishers often follow an hourglass structure [25, 26].

In the first phase (E_u), we identify input differences Δ_i that propagate to low-weight output differences Δ_m with high probability p . The Goldreich–Levin algorithm then efficiently identifies strong output masks Γ_o for the last segment without exhaustive enumeration. We also use many keys throughout all phases to neutralize key-dependent effects and ensure structural distinguishers.

Goldreich–Levin in Brief. We recall succinctly how the Goldreich–Levin (GL) procedure recovers the output mask Γ_o that maximizes the *per-key* linear correlation on the last segment E_ℓ , given a fixed input mask Γ_m . Consider the Boolean oracle

$$f_K(Y) = \langle \Gamma_m, E_{\ell,K}^{-1}(Y) \rangle \in \{0, 1\}.$$

For any mask $A \in \{0, 1\}^{64}$, its Fourier–Walsh coefficient (for key K) is

$$\widehat{f_K}(A) = \mathbb{E}_Y [(-1)^{f_K(Y) \oplus \langle A, Y \rangle}].$$

A direct substitution shows that $\widehat{f_K}(\Gamma_o)$ equals the *per-key* segment correlation q_K of the linear approximation on E_ℓ . The GL task is precisely to recover a mask with a *heavy* coefficient, i.e., $|\widehat{f_K}(A)|$ noticeably above noise.

Algorithmic Sketch (hidden parity with a noise on a 64-bit segment):

- Choose parameters t (subspace dimension), R (iterations), and a detection threshold $\tau > 0$.
- Repeat the following steps R times:
 - Pick a random base point Y_0 and a random t -coordinate mask of Hamming weight t , defining an affine sub-cube $Y_0 \oplus V$, $V \in \{0, 1\}^t$.
 - Evaluate $g(V) = (-1)^{f_K(Y_0 \oplus V)}$ for all 2^t points, and run a fast Walsh–Hadamard transform (FWHT) to obtain the local spectrum $\hat{g}(\alpha)$.
 - Record the indices α with large magnitude, e.g., $|\hat{g}(\alpha)| \geq \tau \cdot 2^t$ (for the unnormalized FWHT). Each such α is the projection of some global mask A onto the chosen coordinates; convert α into per-bit *votes* for the corresponding bits of A .
- Aggregate votes over the R iterations by per-bit majority to obtain a global candidate mask A ; optionally keep the top k candidates.
- Verify each candidate by Monte Carlo: estimate $\text{Cor}(f_K(Y) \oplus \langle A, Y \rangle)$ from fresh samples and across many keys; accept if $|\text{Cor}| \geq \tau$.

Parameter Guidance (choose by signal strength, not block size):

- t (Subspace Dimension): each GL round uses 2^t oracle calls. As a general rule, pick t around or somewhat above $\log_2(1/|q|)$; e.g., if $|q| \approx 2^{-13}$, then $t \approx 16$ is a good starting point.
- R (Iterations): number of random subspaces. Values in the range 8–64 typically suffice for stable per-bit majorities.
- τ (Detection Threshold): report local coefficients only if $|\hat{g}(\alpha)| \gtrsim \tau \cdot 2^t$ (unnormalized). Smaller τ detects weaker signals but requires more verification samples.
- Verification Budget: To confirm $|q| \geq \tau$ with small error, use on the order of $M \approx c/\tau^2$ samples per key (constant c depends on the desired confidence).

Complexity and Practice. GL uses $O(R \cdot 2^t)$ oracle calls and $O(R \cdot t \cdot 2^t)$ time (one FWHT of length 2^t per subspace), versus $\Theta(2^{64})$ for exhaustive search on masks. In our implementation we instantiate f_K with the exact inverse $E_{\ell, K}^{-1}$ and re-estimate the final candidates across many keys to suppress key-dependent artifacts. When no heavy coefficient passes the threshold, we fall back to single-bit masks and select by direct correlation estimation.

6.3 Results on Threefry-2×32

We applied the segmented search on THREEFRY-2×32 and obtained practical differential–linear distinguishers for up to 15 rounds of this 64-bit block cipher. Table 4 summarizes our best results. Note that since the rounds of THREEFRY use different rotation constants, the differential and linear distinguishers depend on the starting round (**offset**) and the chosen segmentation. The first column of Table 4 specifies the number of rounds, the segmentations, as well as the parameters used in Phase L (the Goldreich–Levin algorithm).

Table 4: Specification of DL distinguishers for THREEFRY.

Configuration	Distinguishers	Parameters
Rounds: $1 + 9 + 1 = 11$	Δ_i : 0x82000000, 0x02000000	p : $2^{-1.00}$
Segment: $[3-3] + [4-12] + [13-13]$	Δ_m : 0x80000000, 0x00000000	r : $2^{-4.67}$ (sign +)
Experiments: 2^{16} , Keys: 2^5	Γ_m : 0x00000001, 0x00000000	q : $2^{0.00}$
$t = 12$, $R = 14$, $\tau = 3.125 \times 10^{-2}$	Γ_o : 0x20000001, 0x20000000	Cor: $2^{-5.67}$
Rounds: $2 + 8 + 2 = 12$	Δ_i : 0x80050000, 0x00040000	p : $2^{-3.99}$
Segment: $[0-1] + [2-9] + [10-11]$	Δ_m : 0x80000000, 0x00000000	r : $2^{-6.56}$ (sign -)
Experiments: 2^{16} , Keys: 2^8	Γ_m : 0x00000000, 0x00000040	q : $2^{0.00}$
$t = 12$, $R = 14$, $\tau = 3.1 \times 10^{-2}$	Γ_o : 0x00000001, 0x00000000	Cor: $-2^{-10.54}$
Rounds: $3 + 8 + 2 = 13$	Δ_i : 0x82104020, 0x00100020	p : $2^{-7.55}$
Segment: $[2-4] + [5-12] + [13-14]$	Δ_m : 0x80000000, 0x00000000	r : $2^{-5.94}$ (sign -)
Experiments: 2^{16} , Keys: 2^6	Γ_m : 0x00000000, 0x00000002	q : $2^{-1.00}$
$t = 12$, $R = 14$, $\tau = 3.125 \times 10^{-2}$	Γ_o : 0x60000000, 0x00000000	Cor: $-2^{-15.49}$
Rounds: $3 + 8 + 3 = 14$	Δ_i : 0x82104020, 0x00100020	p : $2^{-7.48}$
Segment: $[2-4] + [5-12] + [13-15]$	Δ_m : 0x80000000, 0x00000000	r : $2^{-5.73}$ (sign -)
Experiments: 2^{23} , Keys: 2^6	Γ_m : 0x00000000, 0x04000000	q : $2^{-1.59}$
$t = 12$, $R = 15$, $\tau = 2.762 \times 10^{-3}$	Γ_o : 0x00808000, 0x00008000	Cor: $-2^{-16.38}$
Rounds: $4 + 8 + 3 = 15$	Δ_i : 0x22884004, 0x02080100	p : $2^{-16.72}$
Segment: $[1-4] + [5-12] + [13-15]$	Δ_m : 0x00000100, 0x00000000	r : $2^{-7.27}$ (sign -)
Experiments: 2^{20} , Keys: 2^6	Γ_m : 0x00000000, 0x00000008	q : $2^{0.00}$
$t = 12$, $R = 15$, $\tau = 2.0 \times 10^{-3}$	Γ_o : 0x01000001, 0x01000000	Cor: $-2^{-23.99}$

The relatively small block size makes it possible to select large middle segments (up to 9 rounds) while keeping the computations in Phase M manageable. All experiments were run on a standard laptop (Apple M4 chip (10 core CPU) in MacBook Pro (Mac16,1)), and the runtime for each configuration ranged from a few minutes to a few hours.

7 Polster: A Replacement for Philox and Threefry

In this section, we introduce a replacement for PHILOX and THREEFRY that provides cryptographic security, while being well suited for PRNGs on GPU architectures. The motivation for this design stems from the need for a cryptographically secure PRNG primitive that maintains high performance in parallel GPU environments, where existing primitives such as PHILOX and THREEFRY may lack strong cryptographic guarantees.

Our replacement is based on a reduced-round variant of the SPECK family of block ciphers [5, 6], specifically a modified version of SPECK-64-128, which has a 64 bit block and a 128 bit key size. SPECK is an Addition-Rotation-XOR based cipher that together with SIMON, an And-Rotation-XOR-based cipher, was designed by NSA as lightweight ciphers optimized for hardware and software implementations. Since their introduction, the cryptographic community

Table 5: The parameters for SPECK family of block ciphers.

block size	$2n$	32	48	48	64	64	96	96	128	128	128
key size	mn	64	72	96	96	128	96	144	128	192	256
number of rounds	R	22	22	23	26	27	28	29	32	33	34

has extensively analyzed these ciphers, which has increased confidence in their security properties.

In the following, we first explain the SPECK block cipher family and describe its encryption algorithm. We then briefly summarize the previous cryptanalysis applied to various versions of SPECK.

Next, we provide the rationale for selecting the specific SPECK variant suitable for our use case. Based on this selection, we present POLSTER, a reduced-round version of the chosen SPECK cipher with a modified key schedule, tailored for high-throughput GPU applications. Finally, we report implementation results and compare the performance of POLSTER with PHILOX and THREEFRY.

7.1 Speck Family of Block Ciphers

SPECK is a family of ARX based block ciphers, designed to provide excellent performance in both hardware and software, with a particular focus on micro-controller environments. The family consists of ten variants, distinguished by their block size and key size parameters. Each cipher operates on a block of two n -bit words (i.e., two branches of n bits), while the key size is composed of m words, with $m \in \{1, 2, 3\}$. Accordingly, a given instance of the cipher is denoted by SPECK- $2n$ - mn , where $2n$ specifies the block length in bits and mn indicates the key length in bits. The recommended number of rounds depends on the specific parameters of the variant. All ten versions of the SPECK family are summarized in Table 5.

The round function of SPECK is conceptually similar to the mixing functions used in the THREEFISH block cipher, and therefore also resembles those of THREEFRY. The key differences are that in SPECK all rounds use an identical function, and a round key is added in every round.

Formally, the round function operates on state $(X_i, Y_i) \in \{0, 1\}^{2n}$ as follows:

$$X_{i+1} = ((X_i \ggg \alpha) + Y_i) \oplus K_i, \quad Y_{i+1} = (Y_i \lll \beta) \oplus X_{i+1},$$

where the addition is modulo 2^n , and (α, β) are the rotation constants. Specifically, $(\alpha, \beta) = (7, 2)$ for $n = 16$ (block size 32 bits), and $(\alpha, \beta) = (8, 3)$ for all larger block sizes.

The key schedule of SPECK uses another ARX round function to generate the round keys K_i . Let K be the master key of a SPECK- $2n$ block cipher. We represent K as $(L_{m-2}, \dots, L_0, K_0)$, where each L_i , and K_0 are n -bit key words and $m \in \{2, 3, 4\}$ denotes the number of words in the key. The sequences of

round keys K_i and intermediate words L_i are then defined iteratively as follows:

$$L_{m+i+1} = (L_i + (L_i \ggg \alpha)) \oplus i, \quad K_{i+1} = (K_i \lll \beta) \oplus L_{m+i+1}.$$

7.2 Cryptanalysis Results on Speck

The designers of SPECK claimed that, although it is a *lightweight* block cipher, it was intended to provide full security for each block and key size against standard chosen-plaintext (CPA) and chosen-ciphertext (CCA) attacks. Resistance against related-key attacks was also considered during the design, though it was treated as a secondary objective, since such attacks are typically not relevant in practical use cases such as PRNGs. On the other hand, no effort was made to protect against known-key distinguishing attacks, nor was SPECK evaluated as a potential hash function.

To date, no successful attack on the full-round version of any SPECK variant has been reported. However, the cipher has attracted significant cryptanalytic attention, and numerous studies have been conducted on its reduced-round variants, including differential [2, 9, 10, 17, 18, 20, 32, 41, 42, 44], linear(-hull) [10, 30], rectangle [2], boomerang [34], integral [22, 29], machine-learning-based distinguishability [16, 28] cryptanalysis. The most effective published attacks on SPECK in the standard attack models are based on differential cryptanalysis. These attacks typically reach about 70–75% of the total rounds for most variants, but they only achieve marginal improvements over brute-force search. According to the designers, differential attacks were identified as the limiting factor during the design process, i.e., the class of attacks capable of penetrating the largest number of rounds. The final choice of the number of rounds was therefore set to ensure a security margin comparable to that of AES-128, at approximately 30% [7].

7.3 Specification of Polster

In the following, we explain the rationale behind selecting a specific SPECK variant that best fits our GPU-based PRNG scenario. Building on this choice, we then introduce POLSTER, specifically optimized for high-throughput GPU implementations while maintaining strong cryptographic properties.

The SPECK family of block ciphers includes variants operating on different word sizes, namely $n \in \{16, 24, 32, 48, 64\}$. Among these, the versions with $n = 32$ and $n = 64$ are the most suitable for GPU architectures. This is because modern GPUs are optimized for 32-bit and 64-bit integer operations, providing native support for modular addition, bitwise rotation, and XOR on these word sizes. As a result, implementations with $n = 32$ or $n = 64$ can be realized with minimal overhead and high throughput, while smaller word sizes would not fully exploit the underlying hardware capabilities, and larger sizes would increase the memory bandwidth requirements without a proportional performance gain.

Block Size. Between the two suitable options, $n = 32$ is typically preferable for GPU-based PRNGs. First, most GPU programming models (such as CUDA and OpenCL) expose 32-bit integer arithmetic as the native data type, with 64-bit operations often incurring additional latency or requiring more hardware resources. Second, GPU memory hierarchies are generally optimized for 32-bit words, meaning that state representation, memory transfers, and coalesced access patterns achieve higher efficiency at this granularity. Finally, since PRNGs for large-scale simulations or machine learning workloads often prioritize parallel throughput over individual stream strength, the reduced per-thread state size of $n = 32$ allows more concurrent threads to be scheduled on the GPU, further improving overall performance.

Key Size. Based on this reasoning, we select SPECK-64 as the candidate block cipher for our GPU-based PRNG. Among the two available key sizes, 96-bit and 128-bit, we prefer the 128-bit key for several reasons. First, a larger key provides a higher security margin against brute-force attacks, which is especially important when the cipher is used as the core of a high-throughput PRNG where large volumes of data are generated. Second, additional key material allows for more flexibility in the key schedule, improving diffusion across the rounds, and reducing potential structural weaknesses. Finally, the performance overhead of using the 128-bit key schedule compared to the 96-bit one is negligible on modern GPUs, since the per-round operations dominate the runtime and are independent of key length. Thus, SPECK-64-128 strikes a practical balance between security, efficiency, and suitability for GPU implementations.

Number of Rounds. The strongest published attack (with respect to the highest number of rounds) on SPECK-64-128 is a differential attack targeting 20 rounds, with a complexity of $2^{125.56}$ encryptions and requiring $2^{61.56}$ chosen plaintexts. This attack, together with many of the other attacks on SPECK, exploits a recurring technical convenience: In numerous cryptanalyses, it is possible to prepend one round to the distinguisher without introducing additional key guesses. This is because adding the round key is equivalent to applying it to the state after all other round operations have been performed. Concretely, for differential cryptanalysis, if a differential distinguisher begins at the second round, we may choose plaintext pairs so that after the first round the state difference equals the input difference of the distinguisher; this allows the distinguisher to be applied to an additional round without guessing any key material.

A similar technique is commonly used in the key-recovery extension of attacks: the right branch at the input of the last round can often be recovered without guessing additional key material. Both of these issues can be mitigated by a simple modification of adding pre-whitening and post-whitening key additions before the first round and after the last round, respectively.

Moreover, the most effective published attacks that reach (or approach) the highest round counts incur very large time and/or data complexity. Consequently, taking into account the whitening countermeasure and the high com-

Algorithm 2: POLSTER Encryption

Input: $P = (X, Y)$ and $K = (K_0, K_1, K_2, K_3)$

```

1  $X \leftarrow X + K_2$  ;
2  $Y \leftarrow Y + K_3$  ;
3  $K_2 \leftarrow K_2 + C_2$  ;
4  $K_3 \leftarrow K_3 + C_3$  ;
5 for  $r = 0$  increasing to  $r = 19$  do
6    $i \leftarrow r \bmod 4$  ;
7    $X \leftarrow ((X \ggg 8) + Y) \oplus K_i$  ;
8    $Y \leftarrow (Y \lll 3) \oplus X$  ;
9    $K_i \leftarrow K_i + C_i$  ;
10  $X \leftarrow X + K_0$  ;
11  $Y \leftarrow Y + K_1$  ;
12 return  $C = (X, Y)$  ;
```

plexity of the best known attacks, we believe that 20 rounds provide a sufficient security margin for our use case.

Key Schedule. Each round function of SPECK consists of two ROT, two XOR, and one ADD operation. Similarly, each update of the key schedule employs exactly the same set and number of operations. This implies that, in a GPU implementation of SPECK, roughly half of the computational cost is incurred by the key schedule if the round keys were not pre-computed at the CPU side and transferred to the GPU memory. Consequently, adopting an alternative key schedule that requires fewer operations per update, while maintaining comparable security, can lead to a significant performance improvement.

We propose using a key schedule similar to the one in PHILOX. Let the user key be $K = (K_0, K_1, K_2, K_3)$, where each K_i is a 32-bit word. For the r -th round, the round key is taken as $K_{r \bmod 4}$, and then updated as

$$K_{r \bmod 4} \leftarrow K_{r \bmod 4} + C_{r \bmod 4},$$

where the addition is modulo 2^{32} and the constants are

$$C_0 = 0\text{xBB67AE85}, C_1 = 0\text{x84CAA73B}, C_2 = 0\text{x9E3779B9}, C_3 = 0\text{x7F4A7C15},$$

which are derived from the constants used in PHILOX. With this approach, the key schedule requires only a single ADD per round, which is significantly lower than the cost of the original SPECK key schedule.

Specification of POLSTER. After introducing the modified key schedule, we present POLSTER, detailed in Algorithm 2. It follows the reduced-round structure of SPECK-64-128 variant while incorporating our lightweight, PHILOX-like key schedule. Note that while round keys are combined with the state using XOR, the whitening keys are applied via modular addition.

Table 6: Performance comparison of POLSTER with PHILOX, THREEFRY, and SPECK. Our results for PHILOX is from the brute force implementation. The reported results are in a unit of [GB/s].

Primitive	MX250	GTX860M	RTX2070 Super	RTX4090	Ref.
PHILOX-4×32-10	50.2	52.8	1800.0	8190.0	[43]
PHILOX-4×32-10	53.9	52.5	1397.9	5517.1	[Ours]
THREEFRY-2×32-20	57.1	48.9	675.0	3057.5	[Ours]
SPECK-64-128	33.4	29.2	302.6	1387.1	[Ours]
POLSTER-64-128	38.7	35.1	327.8	1537.7	[Ours]

7.4 Implementations Results

We used the benchmark⁴ provided by [43] on different GPUs to obtain CTR mode of operation results for PHILOX-4×32-10 and THREEFRY-4×32-20. We observed that on older GPUs, THREEFRY-4×32-20 outperforms PHILOX-4×32-10. However, on modern GPUs its vice versa. This suggests that architectural changes in the future might affect the performance of random number generators and having multiple alternatives might be beneficial.

We provide CTR mode of encryption results of SPECK, POLSTER, PHILOX, and THREEFRY in Table 6. Although PHILOX-4×32-10 and THREEFRY-2×32-20 provide fast results, we already showed in section 4 that their 64-bit key can be practically captured via brute force attacks.

8 Conclusion and Future Work

In this paper, we present the first (and long-overdue) review of the cryptographic strength provided by the most commonly used PRNGs for AI applications. As it turns out, given our brute-force attacks on GPUs and the cryptographic shortcomings of the designs, the most commonly used PRNGs do not provide the level of protection required by modern AI applications. Therefore, we propose POLSTER as a secure and well-established alternative.

While our focus was on providing a well-understood alternative rather than introducing a new, GPU-optimized design, future work might explore the development of such a GPU-optimized, cryptographically strong primitive.

References

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I.J., Harp, A., Irving, G., Isard, M., Jia, Y., Józefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D.G., Olah, C., Schuster, M., Shlens,

⁴ <https://github.com/DEShawResearch/random123>

- J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P.A., Vanhoucke, V., Vasudevan, V., Viégas, F.B., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X.: Tensorflow: Large-scale machine learning on heterogeneous distributed systems. CoRR abs/1603.04467 (2016), <http://arxiv.org/abs/1603.04467>
2. Abed, F., List, E., Lucks, S., Wenzel, J.: Cryptanalysis of the speck family of block ciphers. IACR Cryptol. ePrint Arch. p. 568 (2013), <http://eprint.iacr.org/2013/568>
3. Baidu: Paddlepaddle: Parallel distributed deep learning platform. <https://github.com/PaddlePaddle/Paddle> (2025), accessed: 2025-09-17
4. Bar-On, A., Dunkelman, O., Keller, N., Weizman, A.: DLCT: A new tool for differential-linear cryptanalysis. In: EUROCRYPT 2019. LNCS, vol. 11476, pp. 313–342. Springer (2019)
5. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The SIMON and SPECK families of lightweight block ciphers. IACR Cryptol. ePrint Arch. p. 404 (2013), <http://eprint.iacr.org/2013/404>
6. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The SIMON and SPECK lightweight block ciphers. In: Proceedings of the 52nd Annual Design Automation Conference, San Francisco, CA, USA, June 7–11, 2015. pp. 175:1–175:6. ACM (2015), <https://doi.org/10.1145/2744769.2747946>
7. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: Notes on the design and analysis of SIMON and SPECK. IACR Cryptol. ePrint Arch. p. 560 (2017), <http://eprint.iacr.org/2017/560>
8. Bellini, E., G rault, D., Grados, J., Makarim, R.H., Peyrin, T.: Fully automated differential-linear attacks against ARX ciphers. In: Rosulek, M. (ed.) CT-RSA 2023. LNCS, vol. 13871, pp. 252–276. Springer (2023)
9. Biryukov, A., Roy, A., Velichkov, V.: Differential analysis of block ciphers SIMON and SPECK. In: Cid, C., Rechberger, C. (eds.) Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3–5, 2014. Revised Selected Papers. Lecture Notes in Computer Science, vol. 8540, pp. 546–570. Springer (2014), https://doi.org/10.1007/978-3-662-46706-0_28
10. Biryukov, A., Velichkov, V., Corre, Y.L.: Automatic search for the best trails in ARX: application to block cipher speck. In: Peyrin, T. (ed.) Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20–23, 2016, Revised Selected Papers. Lecture Notes in Computer Science, vol. 9783, pp. 289–310. Springer (2016), https://doi.org/10.1007/978-3-662-52993-5_15
11. Blondeau, C., Leander, G., Nyberg, K.: Differential-linear cryptanalysis revisited. J. Cryptol. 30(3), 859–888 (2017)
12. Borisov, N., Chew, M., Johnson, R., Wagner, D.A.: Multiplicative differentials. In: Daemen, J., Rijmen, V. (eds.) Fast Software Encryption, 9th International Workshop, FSE 2002, Leuven, Belgium, February 4–6, 2002, Revised Papers. LNCS, vol. 2365, pp. 17–33. Springer (2002), https://doi.org/10.1007/3-540-45661-9_2
13. Bradbury, J., Frostig, R., Hawkins, P., Johnson, M.J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., Zhang, Q.: JAX: composable transformations of Python+NumPy programs (2018), <http://github.com/google/jax>
14. Chen, T., Li, M., Li, Y., Lin, M., Wang, N., Wang, M., Xiao, T., Xu, B., Zhang, C., Zhang, Z.: Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. CoRR abs/1512.01274 (2015), <http://arxiv.org/abs/1512.01274>

15. Dahiya, P., Shumailov, I., Anderson, R.: Machine learning needs better randomness standards: Randomised smoothing and prng-based attacks. In: 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association (2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/dahiya>
16. Dani, J., Nakka, K., Saxena, N.: Breaking indistinguishability with transfer learning: A first look at SPECK32/64 lightweight block ciphers. *IACR Cryptol. ePrint Arch.* p. 852 (2024), <https://eprint.iacr.org/2024/852>
17. Dehnavi, S.M.: Further observations on SIMON and SPECK block cipher families. *Cryptogr.* 3(1), 1 (2019), <https://doi.org/10.3390/cryptography3010001>
18. Deng, H., Cao, X., Cheng, Y.: Attention in differential cryptanalysis on lightweight block cipher SPECK. In: 20th Annual International Conference on Privacy, Security and Trust, PST 2023, Copenhagen, Denmark, August 21-23, 2023. pp. 1–9. IEEE (2023), <https://doi.org/10.1109/PST58708.2023.10320201>
19. Dinur, I., Dunkelman, O., Keller, N., Ronen, E., Shamir, A.: Efficient detection of high probability statistical properties of cryptosystems via surrogate differentiation. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023. LNCS, vol. 14007, pp. 98–127. Springer (2023)
20. Dwivedi, A.D., Morawiecki, P.: Differential cryptanalysis in ARX ciphers, application to SPECK. *IACR Cryptol. ePrint Arch.* p. 899 (2018), <https://eprint.iacr.org/2018/899>
21. Dwork, C., Roth, A.: The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.* 9(3-4), 211–407 (2014), <https://doi.org/10.1561/04000000042>
22. Eskandari, Z., Kidmose, A.B., Kölbl, S., Tiessen, T.: Finding integral distinguishers with ease. In: Cid, C., Jr., M.J.J. (eds.) *Selected Areas in Cryptography - SAC 2018 - 25th International Conference*, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers. *Lecture Notes in Computer Science*, vol. 11349, pp. 115–138. Springer (2018), https://doi.org/10.1007/978-3-030-10970-7_6
23. Face, H.: Transformers documentation. <https://huggingface.co/docs/transformers/en/index> (2025), accessed: 2025-09-17
24. Furman, V.: Differential cryptanalysis of nimbus. In: Matsui, M. (ed.) *Fast Software Encryption, 8th International Workshop, FSE 2001 Yokohama, Japan, April 2-4, 2001, Revised Papers*. LNCS, vol. 2355, pp. 187–195. Springer (2001), https://doi.org/10.1007/3-540-45473-X_16
25. Gong, X., Wang, Q., Hao, Y., Jiao, L., Hu, X.: Persistence of hourglass(-like) structure: Improved differential-linear distinguishers for several ARX ciphers. *Cryptology ePrint Archive*, Paper 2025/1667 (2025), <https://ia.cr/2025/1667>
26. Hadipour, H., Derbez, P., Eichlseder, M.: Revisiting differential-linear attacks via a boomerang perspective with application to AES, Ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, Simeck, and SERPENT. In: Reyzin, L., Stebila, D. (eds.) *CRYPTO 2024*. LNCS, vol. 14923, pp. 38–72. Springer (2024)
27. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual* (2020), <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
28. Hou, Z., Ren, J., Chen, S.: Further observations on the security of speck32-like ciphers using machine learning. *PeerJ Comput. Sci.* 11, e3015 (2025), <https://doi.org/10.7717/peerj-cs.3015>

29. Hu, K., Yap, T.: Perfect monomial prediction for modular addition. *IACR Trans. Symmetric Cryptol.* 2024(3), 177–199 (2024), <https://doi.org/10.46586/tosc.v2024.i3.177-199>
30. Huang, M., Wang, L.: Automatic search for the linear (hull) characteristics of ARX ciphers: Applied to speck, sparx, chaskey, and CHAM-64. *Secur. Commun. Networks* 2020, 4898612:1–4898612:14 (2020), <https://doi.org/10.1155/2020/4898612>
31. Jang, E., Gu, S., Poole, B.: Categorical reparameterization with gumbel-softmax. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings. OpenReview.net (2017), <https://openreview.net/forum?id=rkE3y85ee>
32. Jiang, X., Li, M., Makulov, K., Lakhno, V., Sahun, A.: Enhanced neural differential distinguisher for speck32/64 using attention mechanisms and multi cipher-text inputs. *Informatica (Slovenia)* 49(19) (2025), <https://doi.org/10.31449/inf.v49i19.7889>
33. Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., Grill, J., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A.M., Goedeckemeyer, A., Saade, A., Kolesnikov, A., Bendebury, A., Abdagic, A., Vadi, A., György, A., Pinto, A.S., Das, A., Bapna, A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C.L., Choquette-Choo, C.A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D.S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucinska, H., Singh, H., Mehta, H., Lehri, H.T., Hazimeh, H., Ballantyne, I., Szpektor, I., Nardini, I., Pouget-Abadie, J., Chan, J., Stanton, J., Wieting, J., Lai, J., Orbay, J., Fernandez, J., Newlan, J., Ji, J., Singh, J., Black, K., Yu, K., Hui, K., Vodrahalli, K., Greff, K., Qiu, L., Valentine, M., Coelho, M., Ritter, M., Hoffman, M., Watson, M., Chaturvedi, M., Moynihan, M., Ma, M., Babar, N., Noy, N., Byrd, N., Roy, N., Momchev, N., Chauhan, N., Bunyan, O., Botarda, P., Caron, P., Rubenstein, P.K., Culliton, P., Schmid, P., Sessa, P.G., Xu, P., Stanczyk, P., Tafti, P., Shivanna, R., Wu, R., Pan, R., Rokni, R., Willoughby, R., Vallu, R., Mullins, R., Jerome, S., Smoot, S., Girgin, S., Iqbal, S., Reddy, S., Sheth, S., Pöder, S., Bhatnagar, S., Panyam, S.R., Eiger, S., Zhang, S., Liu, T., Yacovone, T., Liechty, T., Kalra, U., Evcı, U., Misra, V., Roseberry, V., Feinberg, V., Kolesnikov, V., Han, W., Kwon, W., Chen, X., Chow, Y., Zhu, Y., Wei, Z., Egyed, Z., Cotruta, V., Giang, M., Kirk, P., Rao, A., Lo, J., Moreira, E., Martins, L.G., Sanseviero, O., Gonzalez, L., Gleicher, Z., Warkentin, T., Mirrokni, V., Senter, E., Collins, E., Barral, J.K., Ghahramani, Z., Hadsell, R., Matias, Y., Sculley, D., Petrov, S., Fiedel, N., Shazeer, N., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Alayrac, J., Anil, R., Lepikhin, D.D., Borgeaud, S., Bachem, O., Joulin, A., Andreev, A., Hardin, C., Dadashi, R., Hussenot, L.: Gemma 3 technical report. *CoRR abs/2503.19786* (2025), <https://doi.org/10.48550/arXiv.2503.19786>
34. Kim, D., Kwon, D., Song, J.: Efficient computation of boomerang connection probability for arx-based block ciphers with application to SPECK and LEA. *IE-*

- ICE Trans. Fundam. Electron. Commun. Comput. Sci. 103-A(4), 677–685 (2020), <https://doi.org/10.1587/transfun.2019EAP1083>
35. Langford, S.K., Hellman, M.E.: Differential-linear cryptanalysis. In: CRYPTO '94. vol. 839, pp. 17–25. Springer (1994)
 36. LeCun, Y.: The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/> (1998)
 37. L'Ecuyer, P., Simard, R.J.: Testu01: A C library for empirical testing of random number generators. ACM Trans. Math. Softw. 33(4), 22:1–22:40 (2007), <https://doi.org/10.1145/1268776.1268777>
 38. Mächtle, F., Serr, J., Loose, N., Sander, J., Eisenbarth, T.: OCEAN: open-world contrastive authorship identification. In: Fischlin, M., Moonsamy, V. (eds.) Applied Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23–26, 2025, Proceedings, Part II. LNCS, vol. 15826, pp. 459–486. Springer (2025), https://doi.org/10.1007/978-3-031-95764-2_18
 39. Mächtle, F., Shetty, A., Sander, J., Loose, N., Pirk, S., Eisenbarth, T.: Prompt pirates need a map: Stealing seeds helps stealing prompts (2025), <https://arxiv.org/abs/2509.09488>
 40. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E.Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Wallach, H.M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E.B., Garnett, R. (eds.) Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada. pp. 8024–8035 (2019), <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
 41. Qin, H., Wu, B.: Towards non-independence of modular additions in searching differential trails of ARX ciphers: new automatic methods with application to SPECK and chaskey. CoRR abs/2203.09741 (2022), <https://doi.org/10.48550/arXiv.2203.09741>
 42. Sajwan, A., Mishra, G.: Comparative analysis of resnet and densenet for differential cryptanalysis of SPECK 32/64 lightweight block cipher. IACR Cryptol. ePrint Arch. p. 1290 (2023), <https://eprint.iacr.org/2023/1290>
 43. Salmon, J.K., Moraes, M.A., Dror, R.O., Shaw, D.E.: Parallel random numbers: as easy as 1, 2, 3. In: Lathrop, S., Costa, J., Kramer, W. (eds.) Conference on High Performance Computing Networking, Storage and Analysis, SC 2011, Seattle, WA, USA, November 12–18, 2011. pp. 16:1–16:12. ACM (2011), <https://doi.org/10.1145/2063384.2063405>
 44. Song, L., Huang, Z., Yang, Q.: Automatic differential analysis of ARX block ciphers with application to SPECK and LEA. In: Liu, J.K., Steinfeld, R. (eds.) Information Security and Privacy - 21st Australasian Conference, ACISP 2016, Melbourne, VIC, Australia, July 4–6, 2016, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9723, pp. 379–394. Springer (2016), https://doi.org/10.1007/978-3-319-40367-0_24
 45. Stability AI: Introducing stable diffusion 3.5. <https://stability.ai/news/introducing-stable-diffusion-3-5> (2024), accessed: 2025-09-17
 46. Tezcan, C.: Optimization of advanced encryption standard on graphics processing units. IEEE Access 9, 67315–67326 (2021), <https://doi.org/10.1109/ACCESS.2021.3077551>

47. Tezcan, C., Leander, G.: GPU assisted brute force cryptanalysis of GPRS, GSM, RFID, and TETRA. *IACR Trans. Symmetric Cryptol.* 2025(1), 309–327 (2025), <https://doi.org/10.46586/tosc.v2025.i1.309-327>
48. Tezcan, C., Leander, G., Hadipour, H.: Cryptanalysis: Theory versus practice: Correcting cryptanalysis results on Ascon, ChaCha, and Serpent using GPUs. *IACR Transactions on Symmetric Cryptology* 2025(3), 729–754 (Sep 2025), <https://tosc.iacr.org/index.php/ToSC/article/view/12484>
49. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA.* pp. 5998–6008 (2017), <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
50. Wang, A., Ai, B., Wen, B., Mao, C., Xie, C., Chen, D., Yu, F., Zhao, H., Yang, J., Zeng, J., Wang, J., Zhang, J., Zhou, J., Wang, J., Chen, J., Zhu, K., Zhao, K., Yan, K., Huang, L., Meng, X., Zhang, N., Li, P., Wu, P., Chu, R., Feng, R., Zhang, S., Sun, S., Fang, T., Wang, T., Gui, T., Weng, T., Shen, T., Lin, W., Wang, W., Wang, W., Zhou, W., Wang, W., Shen, W., Yu, W., Shi, X., Huang, X., Xu, X., Kou, Y., Lv, Y., Li, Y., Liu, Y., Wang, Y., Zhang, Y., Huang, Y., Li, Y., Wu, Y., Liu, Y., Pan, Y., Zheng, Y., Hong, Y., Shi, Y., Feng, Y., Jiang, Z., Han, Z., Wu, Z., Liu, Z.: Wan: Open and advanced large-scale video generative models. *CoRR abs/2503.20314* (2025), <https://doi.org/10.48550/arXiv.2503.20314>
51. Yousefpour, A., Shilov, I., Sablayrolles, A., Testuggine, D., Prasad, K., Malek, M., Nguyen, J., Ghosh, S., Bharadwaj, A., Zhao, J., Cormode, G., Mironov, I.: Opacus: User-friendly differential privacy library in pytorch. *CoRR abs/2109.12298* (2021), <https://arxiv.org/abs/2109.12298>

A Proof of the Distinguishers for Philox

A.1 Proof of Distinguisher 1

Proof. Since $\tilde{R}^{(0)} = R^{(0)} \oplus (2^W - 2)$, and both $R^{(0)}$ and $\tilde{R}^{(0)}$ are odd, we have $\tilde{R}^{(0)} = -R^{(0)}$. Therefore,

$$\begin{aligned}\tilde{L}^{(1)} &= \text{mullo}_M(\tilde{R}^{(0)}) = \text{mullo}_M(-R^{(0)}) = -\text{mullo}_M(R^{(0)}) = -L^{(1)} \\ &= L^{(1)} \oplus (2^W - 2),\end{aligned}$$

with both $L^{(1)}$ and $\tilde{L}^{(1)}$ being odd. Besides,

$$\begin{aligned}\tilde{R}^{(1)} &= \text{mulhi}_M(\tilde{R}^{(0)}) \oplus \tilde{L}^{(0)} \oplus K^{(0)} \\ &= \text{mulhi}_M(-R^{(0)}) \oplus L^{(0)} \oplus \text{mulhi}_M(R^{(0)}) \oplus (M - 1 - \text{mulhi}_M(R^{(0)})) \oplus K^{(0)} \\ &= L^{(0)} \oplus \text{mulhi}_M(R^{(0)}) \oplus K^{(0)} = R^{(1)}.\end{aligned}$$

Thus, after the second round we obtain

$$\tilde{L}^{(2)} = L^{(2)}, \quad \tilde{R}^{(2)} = R^{(0)} \oplus (2^W - 2).$$

Furthermore, we have

$$\begin{aligned}
 \tilde{R}^{(3)} &= \text{mulhi}_M(\tilde{R}^{(2)}) \oplus \tilde{L}^{(2)} \oplus K^{(2)} \\
 &= \text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \oplus L^{(2)} \oplus K^{(2)} \\
 &= \text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \oplus \text{mulhi}_M(R^{(2)}) \oplus R^{(3)}.
 \end{aligned}$$

Since M is odd, for any X , we have $\text{LSB}(\text{mullo}_M(X)) = \text{LSB}(X)$. Therefore,

$$\begin{aligned}
 &\Pr \left[\text{LSB} (L^{(4)} \oplus \tilde{L}^{(4)}) = 0 \right] \\
 &= \Pr \left[\text{LSB} (\text{mullo}_M(R^{(3)}) \oplus \text{mullo}_M(\tilde{R}^{(3)})) = 0 \right] \\
 &= \Pr \left[\text{LSB} (R^{(3)} \oplus \tilde{R}^{(3)}) = 0 \right] \\
 &= \Pr \left[\text{LSB} \left(R^{(3)} \oplus \text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \oplus \text{mulhi}_M(R^{(2)}) \oplus R^{(3)} \right) = 0 \right] \\
 &= \Pr \left[\text{LSB} \left(\text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \oplus \text{mulhi}_M(R^{(2)}) \right) = 0 \right].
 \end{aligned}$$

Next, we analyze the probability that LSB of $\text{mulhi}_M(R^{(2)})$ and LSB of $\text{mulhi}_M(R^{(2)} \oplus (2^W - 2))$ are equal. To this end, we distinguish two cases depending on the parity of $R^{(2)}$: either $R^{(2)}$ is odd, or it is even. Note that each of these cases occur with equal probability.

In the case of $R^{(2)}$ being odd, we have

$$\begin{aligned}
 \text{LSB} \left(\text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \right) &= \text{LSB} \left(\text{mulhi}_M(-R^{(2)}) \right) \\
 &= \text{LSB} \left(M - 1 - \text{mulhi}_M(R^{(2)}) \right) \\
 &= \text{LSB} \left(\text{mulhi}_M(R^{(2)}) \right).
 \end{aligned}$$

Hence, for an odd $R^{(2)}$, we always have $\text{LSB} \left(\text{mulhi}_M(R^{(2)} \oplus (2^W - 2)) \oplus \text{mulhi}_M(R^{(2)}) \right) = 0$.

For the case where $R^{(2)}$ is even, the probability

$$p := \Pr_{X: \text{even}} \left[\text{LSB} \left(\text{mulhi}_M(X \oplus (2^W - 2)) \oplus \text{mulhi}_M(X) \right) = 0 \right]$$

depends on both the word size W and the choice of multiplier M . In the case of PHILOX, all recommended multipliers satisfy that both their MSB and LSB are equal to 1. Under this condition, the above probability equals to

$$p = \frac{M - 2^{W-1} - 1}{2^{W-1}}$$

Combining this with the case where $R^{(2)}$ is odd, we obtain

$$\Pr \left[\text{LSB}(L^{(4)}) = \text{LSB}(\tilde{L}^{(4)}) \right] = \frac{1}{2} \times 1 + \frac{1}{2} \times \frac{M - 2^{W-1} - 1}{2^{W-1}} = \frac{M - 1}{2^W}.$$

For the specific case of PHILOX-2×64, this probability is approximately 0.82.

The above 4-round differential-linear distinguisher can be extended by one round, by using the fact that $\text{mullo}_M(\cdot)$ is a bijection. From the round update, we have

$$\begin{aligned} L^{(4)} &= \text{mulhi}_M(R^{(4)}) \oplus R^{(5)} \oplus K^{(4)} \\ &= \text{mulhi}_M(\text{mullo}_M^{-1}(L^{(5)})) \oplus R^{(5)} \oplus K^{(4)}. \end{aligned}$$

Using definition of F_M , the relation of 4-round distinguisher can be rewritten as

$$\begin{aligned} \Pr \left[\text{LSB}(L^{(4)}) = \text{LSB}(\tilde{L}^{(4)}) \right] &= \\ \Pr \left[\text{LSB}(F_M(L^{(5)}) \oplus R^{(5)}) = \text{LSB}(F_M(L^{(5)}) \oplus R^{(5)}) \right] \end{aligned}$$

which naturally is a 5-round distinguisher on PHILOX-2×W. $\square$

A.2 Proof of Distinguisher 2

Proof. Since $\tilde{R}^{(0)} = R^{(0)}$ and $\tilde{L}^{(0)} = L^{(0)} \oplus 1$, we have $\tilde{L}^{(1)} = L^{(1)}$ and $\tilde{R}^{(1)} = R^{(1)} \oplus 1$. Thus, after the first round, the pair differs only in the LSB of the *right* word. Using the fact that since M is odd, for any X we have $\text{LSB}(\text{mullo}_M(X)) = \text{LSB}(X)$. Thus,

$$\begin{aligned} \text{LSB}(\tilde{L}^{(2)} \oplus L^{(2)}) &= \\ \text{LSB}(\text{mullo}_M(\tilde{R}^{(1)}) \oplus \text{mullo}_M(R^{(1)})) &= \text{LSB}(\tilde{R}^{(1)} \oplus R^{(1)}) = 1. \end{aligned}$$

At the same time,

$$\begin{aligned} \tilde{R}^{(2)} \oplus R^{(2)} &= \\ \text{mulhi}_M(\tilde{R}^{(1)}) \oplus \text{mulhi}_M(R^{(1)}) &= \text{mulhi}_M(R^{(1)} \oplus 1) \oplus \text{mulhi}_M(R^{(1)}). \end{aligned}$$

Consider an integer X with $Q = \text{mulhi}_M(X)$ and $R = \text{mullo}_M(X)$. Accordingly, $X \times M = Q \times 2^W + R$. A standard ± 1 carry/borrow analysis yields the following.

$$(X \oplus 1) \times M = \begin{cases} (X + 1) \times M = Q \times 2^W + R + M & \text{if } X \text{ is even,} \\ (X - 1) \times M = Q \times 2^W + R - M & \text{if } X \text{ is odd.} \end{cases}$$

Then

$$\text{mulhi}_M(X \oplus 1) = \begin{cases} Q + \lfloor (R + M)/2^W \rfloor & \text{if } X \text{ is even,} \\ Q + \lfloor (R - M)/2^W \rfloor & \text{if } X \text{ is odd,} \end{cases}$$

and more precisely,

$$\text{mulhi}_M(X \oplus 1) = \begin{cases} Q + 1 & \text{if } X \text{ is even and } R \geq 2^W - M, \\ Q & \text{if } X \text{ is even and } R < 2^W - M, \\ Q & \text{if } X \text{ is odd and } R \geq M, \\ Q - 1 & \text{if } X \text{ is odd and } R < M. \end{cases}$$

Thus, with probability of $p_{\text{carry}} = (M - 1)/2^W$, we have $\text{mulhi}_M(X \oplus 1) \neq \text{mulhi}_M(Q)$ (with a differing by ± 1), regardless of whether X is even or odd, and with probability $1 - (M - 1)/2^W$, the values are equal.

Next, consider the XOR difference between $\text{mulhi}_M(X)$ and $\text{mulhi}_M(X \oplus 1)$ in the case they are unequal. If Q is a random W -bit word, then

$$Q \oplus (Q + 1) = 2^{\tau_1(Q)+1} - 1, \quad Q \oplus (Q - 1) = 2^{\tau_0(Q)+1} - 1,$$

where $\tau_1(Q)$ (resp. $\tau_0(Q)$) denotes the number of consecutive least significant ones (resp. zeros) in the binary representation of Q .

For a uniformly distributed Q , these values follow a finite geometric distribution: $\Pr[\tau_1 = k] = \Pr[\tau_0 = k] = 2^{-(k+1)}$ for any integer k with $1 \leq k < W$ and $\Pr[\tau_1 = W] = \Pr[\tau_0 = W] = 2^{-W}$. Assuming that $Q = \text{mulhi}_M(X)$ is uniformly distributed over all W -bit integers, we obtain

$$\tilde{R}^{(2)} \oplus R^{(2)} = \begin{cases} 0, & \text{with prob. } 1 - p_{\text{carry}}, \\ 2^d - 1, & \text{with prob. } p_{\text{carry}} \cdot 2^{-d} \text{ for each } d \text{ with } (d \geq 1). \end{cases}$$

In reality, Q is not perfectly uniform over all W -bit integers. However, since the MSB of the chosen multiplier constant M is 1 and W is large, this assumption and the resulting probability estimates remain very close to the true distribution. For the case $W = 32$, and for the recommended values of M , our computations confirm that, except for values of d close to W , the approximation is accurate to high precision. Moreover, as will be shown later, the smaller values of d have a greater impact on the final probability that we need to compute.

Moreover, for the third round, in the right branch we have

$$\begin{aligned} \text{LSB}(\tilde{R}^{(3)} \oplus R^{(3)}) &= \text{LSB}(\text{mulhi}_M(\tilde{R}^{(2)}) \oplus \tilde{L}^{(2)} \oplus \text{mulhi}_M(R^{(2)}) \oplus L^{(2)}) \\ &= \text{LSB}(\text{mulhi}_M(R^{(2)}) \oplus \text{mulhi}_M(\tilde{R}^{(2)})) \oplus 1. \end{aligned}$$

Furthermore, in the fourth round, due to parity preservation by mullo_M , in the left branches we obtain

$$\text{LSB}(\tilde{L}^{(4)} \oplus L^{(4)}) = \text{LSB}(\tilde{R}^{(3)} \oplus R^{(3)}).$$

Therefore,

$$\begin{aligned} \Pr[\text{LSB}(L^{(4)} \oplus \tilde{L}^{(4)}) = 0] &= \Pr[\text{LSB}(R^{(3)} \oplus \tilde{R}^{(3)}) = 0] \\ &= \Pr[\text{LSB}(\text{mulhi}_M(R^{(2)}) \oplus \text{mulhi}_M(\tilde{R}^{(2)})) = 1]. \end{aligned}$$

Based on the possible values for $\tilde{R}^{(2)} \oplus R^{(2)}$, it suffices to consider the cases where $\tilde{R}^{(2)} \oplus R^{(2)} = 2^d - 1$. Hence,

$$\begin{aligned} \Pr[\text{LSB}(L^{(4)} \oplus \tilde{L}^{(4)}) = 0] &= \\ &= p_{\text{carry}} \cdot \sum_{d=1}^W 2^{-d} \cdot \Pr[\text{LSB}(\text{mulhi}_M(R^{(2)}) \oplus \text{mulhi}_M(R^{(2)} \oplus (2^d - 1))) = 1]. \end{aligned}$$

For the case $d = 1$, it can be shown that

$$\Pr \left[\text{LSB} \left(\text{mulhi}_M(R^{(2)}) \oplus \text{mulhi}_M(R^{(2)} \oplus 1) \right) = 1 \right] = \frac{M-1}{2^W}.$$

For cases with $d > 1$, we compute these probabilities using the recommended value of the multiplier in PHILOX-2 $\times$ 32. In this way, we obtain

$$\Pr \left[\text{LSB} \left(L^{(4)} \oplus \tilde{L}^{(4)} \right) = 0 \right] \approx 0.56.$$

Finally, by applying the same approach as in the proof of Distinguisher 1, the above 4-round differential-linear distinguisher can be extended to the 5-round distinguisher mentioned in the statement of the distinguisher. $\square$

A.3 Proof of Distinguisher 3

Proof. To prove this distinguisher, we analyze the difference propagation through each round of PHILOX-4 $\times$ W.

Since $\tilde{X}^{(0)} = X^{(0)}$ and $\tilde{Y}^{(0)} = Y^{(0)}$, it follows directly that $\tilde{X}^{(1)} = X^{(1)}$ and $\tilde{T}^{(1)} = T^{(1)}$. For $Y^{(1)}$ and $\tilde{Y}^{(1)}$, we compute:

$$\begin{aligned} Y^{(1)} &= Z^{(0)} \oplus \text{mulhi}_{M_1}(T^{(0)}) \oplus K_1^{(0)}, \\ \tilde{Y}^{(1)} &= \tilde{Z}^{(0)} \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}) \oplus K_1^{(0)} \\ &= Z^{(0)} \oplus \text{mulhi}_{M_1}(T^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}) \oplus \text{mulhi}_{M_1}(\tilde{T}^{(0)}) \oplus K_1^{(0)} \\ &= Z^{(0)} \oplus \text{mulhi}_{M_1}(T^{(0)}) \oplus K_1^{(0)} = Y^{(1)}. \end{aligned}$$

Moreover, since M_1 is odd and $\text{LSB}(T^{(0)}) = \text{LSB}(\tilde{T}^{(0)})$, we obtain $\text{LSB}(\tilde{Z}^{(1)}) = \text{LSB}(Z^{(1)})$. Hence,

$$\tilde{X}^{(1)} = X^{(1)}, \quad \tilde{Y}^{(1)} = Y^{(1)}, \quad \text{LSB}(\tilde{Z}^{(1)}) = \text{LSB}(Z^{(1)}), \quad \tilde{T}^{(1)} = T^{(1)}.$$

This continues in the second round, where we obtain

$$\tilde{X}^{(2)} = X^{(2)}, \quad \text{LSB}(\tilde{Y}^{(2)}) = \text{LSB}(Y^{(2)}), \quad \tilde{Z}^{(2)} = Z^{(2)}, \quad \tilde{T}^{(2)} = T^{(2)}.$$

Similarly, since M_0 is odd, after the third round we have

$$\text{LSB}(\tilde{X}^{(3)}) = \text{LSB}(X^{(3)}), \quad \tilde{Y}^{(3)} = Y^{(3)}, \quad \tilde{Z}^{(3)} = Z^{(3)}.$$

Therefore, after the fourth round it follows that

$$\tilde{X}^{(4)} = X^{(4)}, \quad \text{LSB}(\tilde{T}^{(4)}) = \text{LSB}(T^{(4)}).$$

Finally, since M_0 is odd, after the fifth round we obtain

$$\text{LSB}(\tilde{Z}^{(5)}) = \text{LSB}(Z^{(5)}).$$

Similar to the case of PHILOX-2×W, the above 5-round differential-linear distinguisher on PHILOX-4×W can be extended by exploiting the fact that $\text{mullo}_{M_1}(\cdot)$ is bijection. From the round update we obtain

$$\begin{aligned} Z^{(5)} &= \text{mulhi}_{M_1}(T^{(5)}) \oplus Y^{(6)} \oplus K_1^{(5)} \\ &= \text{mulhi}_{M_1}(\text{mullo}_{M_1}^{-1}(Z^{(6)})) \oplus Y^{(6)} \oplus K_1^{(5)} \\ &= F_{M_1}(Z^{(6)}) \oplus Y^{(6)} \oplus K_1^{(5)}. \end{aligned}$$

Consequently, the distinguisher condition translates to

$$\text{LSB}(F_{M_1}(\tilde{Z}^{(6)}) \oplus \tilde{Y}^{(6)}) = \text{LSB}(F_{M_1}(Z^{(6)}) \oplus Y^{(6)}).$$

Thus, the distinguisher naturally extends to a 6-round differential-linear distinguisher on PHILOX-4×W. $\square$